003: 自社システム編 — SaaSを潰す

> **このフェーズの本質**: BPO編で「SaaSを繋いで自動化」していた世界から、
> 「そもそもSaaS要らなくない？」という世界に進化させる。
> 月5万のkintone + 月3万のfreee + 月2万のジョブカン = 月10万を、
> シャチョツーの中に全部統合してゼロにする。
> SIerに月50万払う世界も終わらせる。

このドキュメントの使い方（SMB / Enterprise 別読み方ガイド）

	SMB（10〜200名）	Enterprise（200名以上・上場等）
主目標	月額SaaS費用を削減する	基幹システム・ERPをAIで内製化
移行アプローチ	ブラウザ拡張で透明移行（§4.5）	レガシーシステム統合 + CAB承認（§6.5）
優先フェーズ	Phase 6（ROI高い業務から）	Phase 6〜7全て
スキップ可	§4.1 Monte Carlo ROI（概算でも可）・§6.5 Enterprise部分	なし

SMB向け — SaaS解約の最短ルート

```
Step 1: SaaS解約ロードマップ自動生成（§4.7）
  → 現在の月額SaaS費用を入力
  → 依存関係グラフから「解約できる順番」を自動算出
  → 例: Slack→LINE WORKS移行（月3万円削減）から開始

Step 2: ブラウザ拡張で既存SaaSをUI複製（§4.5）
  → Phase A: 既存SaaSと並行して動作
  → Phase B: 拡張機能で業務UXを再構築（フロー統合・自動化・最適化）
  → Phase C: 完全カットオーバー（SaaS解約）

Step 3: 自動保守パイプライン（§4.6）
  → ナレッジ変更時にシステムを自動パッチ
  → 「社長が方針変更 → 翌日にシステム反映」

SMBで最初に解約すべきSaaS候補:
  1. 汎用タスク管理（Asana/Notion → 自社ダッシュボードへ）
  2. 勤怠管理（汎用SaaS → 自社特化システムへ）
  3. 社内申請フォーム（Googleフォーム等 → 承認フロー内製化）
```

Enterprise向け — SMB機能に加えて実装（§6.5参照）

```
◎ レガシーシステム統合（§6.5 パターン1〜4）
  → API連携 / DBダイレクト / ファイル連携 / iPaaS
  → SAP・Salesforce等の基幹システムとの接続

◎ CAB承認フロー（Change Advisory Board）
  → 生成コードのセキュリティレビュー → リリース承認
  → IT部門・情報セキュリティ部門を承認プロセスに組み込み

◎ 展開オプション A〜D（§6.5）
  → Option A: 共有クラウド（SMBデフォルト）
  → Option B: 専用インスタンス（中規模Enterprise）
  → Option C: 顧客GCPプロジェクト内展開
  → Option D: オンプレミス（金融・医療）

◎ Composable System Architecture（§4.2）
  → 原子コンポーネント（データ/フロー/計算/AI型）の組み合わせ
  → 業務変化に対してコンポーネント単位で差し替え可能
  → 大規模システムの保守コスト大幅削減

SLA:
  SMB: Standard 99.9%（月8.7時間停止許容）
  Enterprise: Premium 99.95%（月4.4時間停止許容）
```

---

1. 潰す対象の構造理解

```
■ SaaSが売っているもの

  ① 業務機能          kintone（DB/ワークフロー）, freee（会計）, ジョブカン（勤怠）
  ② データの保管場所   「御社のデータはクラウドに安全に保管」
  ③ 業務プロセスの型   「ベストプラクティスに沿ったワークフロー」
  ④ SaaS間連携        「freeeとSlackを連携できます」

■ SIerが売っているもの

  ① カスタム開発       「御社専用のシステムを作ります」（500万-3,000万）
  ② 保守運用           「月50万でサーバー管理します」
  ③ ヘルプデスク       「困ったら電話ください」
  ④ ベンダーロックイン 「このシステムは弊社しか触れません」

■ これらの弱点（= シャチョツーの勝ち筋）

  SaaS:
    ・高い積み上げ（kintone + freee + Slack + ... = 月10-30万）
    ・データが分散（各SaaSに情報がバラバラ）
    ・連携が面倒（iPaaS設定は素人には無理）
    ・「うちのやり方」に合わない（SaaSの型に業務を合わせている）
    ・SaaS疲れ（増えるほど管理コスト増大）

  SIer:
    ・高い（初期500万〜 + 保守月50万〜）
    ・遅い（要件定義3ヶ月→開発6ヶ月→テスト2ヶ月 = 11ヶ月）
    ・変更できない（仕様変更 = 追加見積もり）
    ・属人的（担当SE退職 = ブラックボックス化）
    ・ベンダーロックイン（他社に乗り換えられない）

■ シャチョツーの圧倒的優位

  ・SaaS統合（バラバラのSaaSを1つのBrainに統合済み）
  ・データ一元化（BPO編で全データがBrainに流れ込んでいる）
  ・「うちのやり方」が標準（ナレッジ = 業務ロジックそのもの）
  ・変更が即時（ナレッジを変更 → システムが自動適応）
  ・要件定義が不要（BPO編の実行データ = 要件定義の素材）
  ・開発速度が圧倒的（Engineer Worker + develop_agent）
```

2. 自社システム編のゴール

```
入口: BPO編の実行データ（どの業務がどう自動化されたか）
 ↓
分析: 「この部分、毎回SaaSに繋いで処理してるけど、
        シャチョツーの中にシステム作った方が速くない？」
 ↓
対話: Brain ↔ Engineer Worker（develop_agent）で要件定義
 ↓
構築: develop_agent が設計→実装→テスト→デプロイ
 ↓
出口: 社長が必要としていたシステムが、
      SIerの1/10の費用と1/5の期間で完成する
      → 「もうkintoneもfreeeもSIerもいらないじゃん」
```

KPI（SaaS/SIer代替の証明）

指標	目標	SIer比較
要件定義の速度	対話5ラウンド（1-2日）	SIer: 1-3ヶ月
開発期間	1-4週間	SIer: 3-11ヶ月
開発コスト	従量課金内（追加10-30万）	SIer: 500-3,000万
仕様変更の速度	ナレッジ変更で即時	SIer: 追加見積もり2週間
削減SaaS月額	月5-20万削減	—

---

3. 実装フェーズ

Phase 5（後半）: Engineer Worker + 要件定義AI（Phase 5のBPO Level 3-4と並行）

```
■ Engineer Worker = Brain とdevelop_agent の対話で要件定義書を作る

  [Brain側: ビジネス知識]
    ・蓄積されたナレッジ（業務フロー・判断基準・暗黙知）
    ・BPO実行データ（どの業務がどう処理されていたか）
    ・自動化不可能ステップの情報（コネクタの限界）
    ・ROIデータ（どの業務改善が最もインパクト大きいか）

  [develop_agent側: 技術知識]
    ・技術的な実現可能性の判定
    ・コスト・期間の見積もり
    ・過去の類似プロジェクトの実績（ネットワーク知性から）
    ・アーキテクチャ提案

  [対話フロー]
    Round 1: Brain → develop_agent 「こういう課題がある」
    Round 2: develop_agent → Brain 「技術的にはこう解決できる。質問がある」
    Round 3: Brain → develop_agent 「ナレッジを確認した。回答はこう」
    Round 4: develop_agent → Brain 「このスコープで要件定義書を作る」
    Round 5: 合意 → 共通要件定義書を生成

  [BPOデータからの要件自動抽出]（★追加設計）
    BPO実行ログから:
      ・「毎月kintone→freee転記を繰り返している」
        → 提案: 「受注→請求の一体型システムを作りませんか？
                 SaaS2つ分の月額¥8万が不要になります」
      ・「Excel日報を毎日処理している」
        → 提案: 「専用の日報入力画面を作りませんか？
                 入力時間が1/3になり、自動集計も付きます」
```

タスク:

- [ ] Brain ↔ develop_agent 対話マネージャ
- [ ] 要件定義書ビルダー（対話結果 → 要件定義書生成）
- [ ] 情報不足検知 → 社長に質問 → 補完パイプライン
- [ ] BPO実行ログからの「システム化候補」自動検出（★4.1参照）
- [ ] コスト・期間見積もりエンジン（ネットワーク知性から類似事例参照）
- [ ] develop_agent への要件定義書入力 → 実装パイプライン接続

完了基準:
- 要件定義書が対話形式で作成できる
- 情報不足時は社長に自動で質問が飛ぶ
- BPO実行データから「システム化した方がいい業務」が自動提案される

Phase 6（一部）: SaaS段階的置換 + セキュリティ強化

```
■ SIer段階的置換戦略（02_プロダクト設計.md 15.2 を実行フェーズに展開）

  Phase 1: 共存（0-2年目）— SIerの領域外で始める
  ──────────────────────────────────────
    シャチョツー = LINE/メール/Excel/Q&A（SIerと被らない）
    SIer = サーバー/ネットワーク/基幹システム（そのまま）
    → SIerとは友好関係。紹介プログラム15%で味方にする。

  Phase 2: 侵食（2-4年目）— SIerの周辺業務を取る
  ──────────────────────────────────────
    ・データバックアップ → シャチョツーが自動化済み
    ・SaaS間連携 → コネクタで自動化済み
    ・社内ヘルプデスク → AIが回答
    ・Excel→SaaS移行支援 → シャチョツー内で完結
    → 顧客が「SIerに月50万払う理由なくない？」と気づく

  Phase 3: 置換（4-7年目）— SIerの存在意義がなくなる
  ──────────────────────────────────────
    ・Engineer Worker が個社専用システムを自動構築
    ・セキュリティ・監視もシャチョツーが一元管理
    ・「IT部門」の機能をシャチョツーが代替
    → SIerの契約解除

■ SaaS段階的置換（個社単位での判断）

  判定: BPOデータから「このSaaSを置き換えるべきか」を自動分析

    置換すべき条件:
      ① 月額コスト > ¥30,000（効果が大きい）
      ② 利用機能が20%以下（全機能の一部しか使っていない）
      ③ BPO連携で頻繁にエラー（APIの不安定さ）
      ④ データがシャチョツー側に全て存在（二重持ちの無駄）

    置換すべきでない条件:
      ① 法令で特定ソフトが必須（電子申告等）
      ② 取引先指定のツール（ANDPAD等、元請が指定）
      ③ 深い専門機能を使用（会計の仕訳AIなど）
```

---

4. 詰めるべき設計ポイントと提案

4.1 BPO実行ログ → システム化候補自動検出 — 結晶化ポイント分析

課題: Engineer Workerは「社長が相談してきた時」に動く受動的な設計。BPOデータを分析して「ここ、システム作った方がいいですよ」と能動的に提案する仕組みがない。単純なパターンマッチング（高頻度タスク検出等）では「点」でしか捉えられず、「複数のBPOタスクが常にセットで実行され、データを共有し、1つのシステムに結晶化すべきクラスタ」を見逃す。

なぜ重要か: BPOは「既存SaaSを使って自動化する」世界。しかし本当のコスト削減は「SaaS自体を不要にする自社システム」から生まれる。BPO実行ログには「どのタスク群が常にセットで動くか」「どのSaaS間でデータが行き来するか」のトポロジーが隠れている。このトポロジーを分析すれば、**「この5つのBPOタスクは1つのシステムに統合すべき」**という結晶化ポイントが自動的に浮かび上がる。

■ 提案: BPO実行トポロジー分析 + モンテカルロROI試算 + 成熟度モデル

[STEP 1] BPO実行トポロジーの構築（タスク共起グラフ）

BPO実行ログから「タスク共起グラフ」を自動構築する。
ノード = BPOタスク、エッジ = 共起関係（同一ワークフロー内 or 24h以内に連続実行）

エッジの重み計算:

```
edge_weight(task_A, task_B) =
  0.40 × co_execution_freq      -- 同時実行頻度（月間）
+ 0.30 × data_sharing_score     -- 共有データフィールド数 / 全フィールド数
+ 0.20 × temporal_proximity     -- 平均実行間隔の近さ（1h以内=1.0, 24h=0.3）
+ 0.10 × error_correlation      -- 片方のエラーが他方に影響する頻度
```

例: ある建設会社の実行ログ（直近3ヶ月）

```
┌──────────────────────────────────────────────────────────┐
│  タスク共起グラフ（エッジの太さ = 重み）                    │
│                                                          │
│  [kintone受注取得] ─── 0.92 ─── [freee請求書作成]          │
│        |                              |                  |
│      0.85                           0.78                 |
│        |                              |                  |
│  [原価データ集計] ─── 0.88 ─── [利益率計算]               |
│        |                              |                  |
│      0.45                           0.72                 |
│        |                              |                  |
│  [日報集計]                      [請求書PDF生成]          |
│                                       |                  |
│                                     0.91                 |
│                                       |                  |
│                                 [メール送付]              |
└──────────────────────────────────────────────────────────┘
```

[STEP 2] 密部分グラフ（結晶化ポイント）の検出

グラフから「密なクラスタ」を検出する。各クラスタ = システム化候補。

アルゴリズム:

1. エッジ重み ≥ 0.6 のサブグラフを抽出
2. Louvain法でコミュニティ検出（modularity最大化）
3. 各コミュニティに対して密度スコアを計算:
   cluster_density = Σ(edge_weights) / (n × (n-1) / 2)
   ※ n = クラスタ内ノード数
4. cluster_density ≥ 0.7 のクラスタ = 「結晶化ポイント」

上記グラフの場合:

```
結晶化ポイント1: {kintone受注取得, freee請求書作成, 原価データ集計,
                  利益率計算, 請求書PDF生成, メール送付}
  → density: 0.81 → 結晶化候補
```

```
      → 提案: 「受注→原価→請求→送付」を一体管理するシステム

    結晶化ポイント2: {日報集計} (孤立ノード)
      → density: N/A → 単独タスクとして残す（BPOで十分）

[STEP 3] 関連SaaS・データフローの特定
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

  結晶化ポイント内のタスクが使っているSaaSとデータを分析:

  結晶化ポイント1の分析結果:
    使用SaaS: kintone（受注管理app_id:15）, freee（請求書）
    データフロー:
      kintone.受注レコード → [会社名, 金額, 工事名, 担当者]
        → freee.請求書下書き → [取引先, 金額, 品目, 税率]
          → PDF生成 → メール送付
    共有データフィールド: 会社名↔取引先, 金額, 工事名↔品目（12フィールド中8共有）
    現在のSaaS月額: kintone ¥5,000 + freee ¥3,000 = ¥8,000
    月間実行回数: 平均15回
    平均処理時間: 42分/回（BPO経由）
    エラー率: 12%（主に金額不一致）

[STEP 4] モンテカルロROIシミュレーション
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

  単純な「月額削減額」ではなく、複数シナリオの確率分布でROIを試算する。

  入力パラメータ（各々に分布を設定）:
    ・月間実行回数: Normal(μ=15, σ=3)        -- 季節変動考慮
    ・1回あたり削減時間: Uniform(20min, 40min) -- 最小〜最大
    ・時間単価: ¥3,000（固定）
    ・SaaS月額削減: ¥8,000（固定）
    ・エラー削減率: Beta(α=8, β=2)           -- 高い確率で70-90%削減
    ・エラー1件あたりの修正コスト: Lognormal(μ=¥5,000, σ=¥2,000)
    ・開発コスト: Triangular(¥30,000, ¥50,000, ¥80,000)
    ・保守コスト: ¥5,000/月（固定）

  シミュレーション（10,000回試行）:
    monthly_benefit = (exec_count × time_saved × hourly_rate)
                    + saas_saving
                    + (exec_count × error_rate_reduction × error_fix_cost)
                    - maintenance_cost

    payback_months = dev_cost / monthly_benefit

  出力:
    ┌──────────────────────────────────────────────────────────┐
    │  ROIシミュレーション結果（10,000回試行）                  │
    ├──────────────────────────────────────────────────────────┤
    │                                                          │
    │  月間効果:                                               │
    │    P10（悲観）: ¥18,200/月                               │
```

```
      │    P50（中央）: ¥27,500/月                          │
      │    P90（楽観）: ¥38,800/月                          │
      │                                                      │
      │  投資回収期間:                                       │
      │    P10（悲観）: 4.4ヶ月                              │
      │    P50（中央）: 1.8ヶ月                              │
      │    P90（楽観）: 0.8ヶ月                              │
      │                                                      │
      │  3ヶ月以内にROI達成する確率: 87%                    │
      │  6ヶ月以内にROI達成する確率: 99%                    │
      │                                                      │
      │  ※ 類似企業12社の実績データで校正済み               │
      └──────────────────────────────────────────────────────┘

  [STEP 5] システム提案の成熟度モデル（4段階ゲート）
  ═══════════════════════════════════════════

    提案は段階的に成熟させる。各ゲートに具体的な通過条件を設定:

    Stage 1: proposal（自動検出）
      通過条件:
        ・結晶化ポイントの密度 ≥ 0.7
        ・月間タスク実行回数 ≥ 10
        ・関連SaaSが2つ以上 OR エラー率 ≥ 10%
      出力: 社長への初回提案（概算ROI付き）

    Stage 2: validated（社長承認 + データ検証）
      通過条件:
        ・社長が「検討する」を選択
        ・直近1ヶ月のデータで再検証し、ROI推定値の誤差 ≤ 20%
        ・類似企業での開発実績 ≥ 3社 OR 社長が「実績なしでもOK」
      出力: 詳細分析レポート

    Stage 3: spec_ready（要件定義完了）
      通過条件:
        ・Engineer Worker ↔ Brain の要件定義対話が完了（5ラウンド以上）
        ・全データフィールドのマッピングが確定
        ・移行対象SaaSのデータエクスポート方法が確認済み
        ・テスト条件（入力→期待出力）が10件以上定義済み
      出力: 要件定義書 + テスト仕様

    Stage 4: development_ready（開発着手可能）
      通過条件:
        ・使用コンポーネント（→ 4.2参照）が全て利用可能
        ・ステージング環境が準備済み
        ・移行計画（→ 4.3参照）が策定済み
        ・社長の最終承認
      出力: 開発チケット + 移行計画書

■ LangGraph 状態遷移

  class CrystallizationState(TypedDict):
```

```
      company_id: str
      execution_topology: dict           # タスク共起グラフ
      crystallization_points: list[dict] # 検出されたクラスタ群
      current_analysis: dict             # 分析中のクラスタ
      saas_mapping: dict                 # クラスタ内SaaS・データフロー
      monte_carlo_result: dict           # ROIシミュレーション結果
      maturity_stage: str                #
proposal/validated/spec_ready/development_ready
      gate_checks: dict                  # 各ゲートの通過状況
      user_decisions: list[dict]         # 社長の意思決定履歴

  グラフ:
    build_topology → detect_clusters → analyze_saas_flow
        → simulate_roi → evaluate_gate → present_proposal
        ↓ (社長承認)
    validate_with_data → requirement_dialogue → check_spec
        → prepare_development → final_approval
        ↓ (各段階で社長に確認)
    (却下) → record_rejection → adjust_thresholds

■ DB スキーマ

  -- 結晶化ポイント（BPOタスク群のクラスタ）
  CREATE TABLE system_crystallization_points (
    id UUID PRIMARY KEY,
    company_id UUID REFERENCES companies(id),
    cluster_name TEXT NOT NULL,                -- LLMが自動命名（例: 「受注→請求一体化」）
    task_ids UUID[] NOT NULL,                  -- クラスタに含まれるBPOタスクID
    cluster_density NUMERIC(4,3) NOT NULL,     -- 密度スコア (0.0-1.0)
    edge_weights JSONB NOT NULL,               -- タスクペアごとの共起重み
    involved_saas TEXT[] NOT NULL,             -- 関連SaaS名リスト
    data_flow_map JSONB NOT NULL,              -- SaaS間データフローの詳細
    shared_fields_count INTEGER,               -- 共有データフィールド数
    monthly_execution_count INTEGER,           -- 月間実行回数
    current_error_rate NUMERIC(4,3),           -- 現在のエラー率
    current_monthly_cost INTEGER,              -- 現在の月額コスト（SaaS + 工数）
    monte_carlo_result JSONB,                  -- {p10, p50, p90, roi_prob_3m,
roi_prob_6m}
    maturity_stage TEXT DEFAULT 'proposal',    --
proposal/validated/spec_ready/development_ready
    gate_check_log JSONB DEFAULT '[]',         -- 各ゲート通過時の検証結果
    similar_projects_count INTEGER DEFAULT 0,
    status TEXT DEFAULT 'active',              -- active/accepted/rejected/completed
    rejection_reason TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
  );

  -- 成熟度ゲート通過ログ
  CREATE TABLE crystallization_gate_log (
    id UUID PRIMARY KEY,
    crystallization_id UUID REFERENCES system_crystallization_points(id),
```

```
    from_stage TEXT NOT NULL,
    to_stage TEXT NOT NULL,
    gate_checks JSONB NOT NULL,               -- 各チェック項目の合否
    approved_by UUID REFERENCES users(id),
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
  );
```

4.2 Composable System Architecture — アトミックコンポーネントによるシステム生成

課題: 「5つの開発パターン（A-E）」では粒度が粗すぎる。実際の中小企業システムは「データ入力+承認フロー+帳票出力+通知」のように複数パターンを組み合わせる。パターンの組み合わせ爆発に対応できない。さらに、個社の業務ルール（ナレッジ）をシステムにどう注入するかの設計がない。

なぜ重要か: SIerの弱点は「毎回ゼロからカスタム開発する」こと。シャチョツーの優位は「小さな部品の組み合わせで高速にシステムを構築する」こと。レゴブロックのように標準化されたアトミックコンポーネントを持ち、要件定義結果に応じてLLMがコンポーネントを組み合わせてシステムを生成する。個社固有のロジックは「ビジネスロジックプラグイン」として汎用コンポーネントに注入する。これにより、SIerの1/10の期間・1/10のコストでカスタムシステムが量産できる。

■ 提案: アトミックコンポーネントライブラリ + コンポジションエンジン + ビジネスロジックプラグイン

[1] アトミックコンポーネントライブラリ

各コンポーネントは「1つの責務」を持つ。全てに標準インターフェースを定義。

■ データ系コンポーネント
```
data_input:         フォーム入力（テキスト/数値/日付/画像/ファイル）
data_list:          一覧表示（ソート/フィルタ/ページネーション）
data_detail:        詳細表示（1レコード）
data_import:        CSV/Excelインポート（バリデーション付き）
data_export:        CSV/Excel/PDFエクスポート
data_sync:          外部SaaSとの双方向同期
```

■ フロー系コンポーネント
```
approval_flow:      承認ワークフロー（N段承認/条件分岐/委任）
notification:       通知（LINE/Slack/メール/SMS）
scheduler:          スケジュール実行（Cron/イベントトリガー）
state_machine:      状態遷移管理（カスタム状態定義可能）
```

■ 計算系コンポーネント
```
calculation_engine:  数式計算（四則演算/条件分岐/集計関数）
report_generator:    レポート生成（集計/グラフ/比較分析）
pdf_builder:        帳票PDF生成（テンプレート差し込み）
dashboard_widget:    ダッシュボード部品（KPI/グラフ/アラート）
```

■ AI系コンポーネント
```
llm_classifier:     LLM分類（テキスト→カテゴリ）
llm_extractor:      LLM構造化（テキスト→JSON）
anomaly_detector:    異常検出（統計ベース+LLMハイブリッド）
```

各コンポーネントの標準インターフェース:
```
{
  "component_id": "approval_flow_v2",
  "input_schema": {
    "item": {"type": "object", "description": "承認対象のデータ"},
    "requester_id": {"type": "string"},
    "approval_rules": {"type": "object", "description": "承認ルール定義"}
  },
  "output_schema": {
    "result": {"type": "string", "enum": ["approved", "rejected", "pending"]},
    "approved_by": {"type": "string"},
    "comments": {"type": "array"}
  },
  "config_schema": {
    "stages": {"type": "integer", "default": 1, "max": 5},
    "timeout_hours": {"type": "integer", "default": 24},
    "escalation_rules": {"type": "array"},
    "delegation_enabled": {"type": "boolean", "default": false}
  },
```

```
    "dependencies": ["notification"],
    "avg_setup_time_hours": 2,
    "reliability_score": 0.97
  }

[2] システム = コンポーネントコンポジショングラフ
═══════════════════════════════════════

  要件定義書をLLMが解析し、コンポーネントの組み合わせグラフを生成する。

  分解フロー:
    1. 要件定義書 → LLMが「必要な機能」をリストアップ
    2. 各機能 → 最適なコンポーネントにマッチング（Embedding類似度）
    3. コンポーネント間のデータフローを定義（output → input の接続）
    4. 足りないコンポーネント → 新規生成パイプライン（→ [3]参照）

  具体例: 「受注→請求一体化システム」の分解

    要件: 受注データ入力 → 原価計算 → 利益率チェック → 承認
          → 請求書PDF生成 → メール送付 → 月次集計ダッシュボード

    コンポジショングラフ:
      ┌──────────┐    ┌──────────────┐    ┌──────────────┐
      │ data_input │ → │ calculation  │ → │ anomaly      │
      │ (受注情報) │    │ _engine      │    │ _detector    │
      │            │    │ (原価+利益率)│    │ (利益率警告) │
      └──────────┘    └──────────────┘    └──────────────┘
            │                                    │
            ↓                                    ↓
      ┌──────────┐    ┌──────────────┐    ┌──────────────┐
      │ data_list  │    │ approval     │ ← │ notification │
      │ (受注一覧) │    │ _flow        │    │ (LINE通知)   │
      └──────────┘    │ (経理部長)   │    └──────────────┘
                        └──────────────┘
                              │
                              ↓
      ┌──────────┐    ┌──────────────┐    ┌──────────────┐
      │ dashboard │ ← │ pdf_builder  │ → │ notification │
      │ _widget   │    │ (請求書)     │    │ (メール送付) │
      │ (月次集計)│    └──────────────┘    └──────────────┘
      └──────────┘

    コンポーネント数: 8個
    推定組立時間: 3日（テンプレートあり）〜7日（カスタム多い場合）
    ※ SIerが同じシステムを作ると: 3-6ヶ月 / ¥500-1,500万

[3] コンポーネント生成パイプライン（ライブラリにないものを作る）
═══════════════════════════════════════

  既存コンポーネントでカバーできない機能は、AIがコードを生成する。
  ただし安全性を担保するための5段階チェックを必須とする。
```

```
    生成パイプライン:
      Step 1: 要件定義 → LLMがコンポーネント仕様を生成
         ・input_schema / output_schema / config_schema を先に定義
         ・既存の類似コンポーネントがあれば、差分のみ生成

      Step 2: コード生成 → Claude API でコード実装
         ・テンプレートコード（Pythonバックエンド + Next.jsフロント）をベースに
         ・生成されるコードは必ず「サンドボックス内で実行可能」な形式

      Step 3: 静的解析（AST検証 + セキュリティスキャン）
         ・ASTパース: 構文的に正しいか
         ・危険パターン検出: eval() / exec() / os.system() / subprocess
         ・SQLインジェクション検出: 生SQLの有無
         ・依存ライブラリ検証: 許可リストに含まれるか
         ・RLS違反検出: company_idフィルタの欠如

      Step 4: サンドボックス実行テスト
         ・Docker コンテナ内で実行（ネットワーク隔離）
         ・テストケース（要件定義時に定義済み）を自動実行
         ・パフォーマンスベンチマーク: レスポンス ≤ 500ms / メモリ ≤ 256MB
         ・エッジケーステスト: 空入力 / 巨大データ / 不正入力

      Step 5: 人間レビュー（セキュリティ + ビジネスロジック）
         ・自動チェック全通過のコードのみ人間レビュー対象
         ・レビュー観点: データの流出リスク / テナント分離 / 計算精度
         ・レビュー通過 → ライブラリに登録（再利用可能に）

[4] ビジネスロジックプラグイン — 個社ナレッジの注入
═══════════════════════════════════════════

    汎用コンポーネントに個社固有のルールを「プラグイン」として注入する。
    コンポーネントのコードを変更せず、設定で挙動を変える。

    プラグインの種類:
      ① バリデーションプラグイン
         例: data_input コンポーネントに「金額は10万以上1億以下」を注入
         ソース: knowledge_items の「値決めルール」

      ② 計算式プラグイン
         例: calculation_engine に「諸経費率 = 材料費の25%（ただし1000万超は20%）」を注入
         ソース: knowledge_items の「原価計算ルール」

      ③ 分岐条件プラグイン
         例: approval_flow に「500万超は社長承認、それ以外は部長承認」を注入
         ソース: knowledge_items の「承認基準」

      ④ 表示テンプレートプラグイン
         例: pdf_builder に「自社ロゴ・住所・振込先」を注入
         ソース: companies テーブルの基本情報

    プラグインの自動生成:
      ナレッジ → LLMが構造化 → プラグインJSON → コンポーネントに適用
```

```
      {
        "plugin_type": "calculation",
        "target_component": "calculation_engine",
        "rules": [
          {
            "name": "諸経費率計算",
            "formula": "IF(material_cost > 10000000, material_cost * 0.20, material_cost
* 0.25)",
            "source_knowledge_id": "KI_045",
            "confidence": 0.92
          }
        ]
      }

    ナレッジが変更されたら:
      ・プラグインが自動更新される
      ・影響範囲を計算（どのシステムのどの画面に影響するか）
      ・影響が大きい場合は社長に通知 + 確認

■ LangGraph 状態遷移

  class SystemCompositionState(TypedDict):
      company_id: str
      spec_document: dict                # 要件定義書
      required_features: list[str]       # LLMが抽出した必要機能リスト
      component_mapping: dict            # 機能 → コンポーネントのマッピング
      composition_graph: dict            # コンポーネント間の接続グラフ
      missing_components: list[dict]     # ライブラリにないコンポーネント
      generation_pipeline: dict          # コンポーネント生成の進捗
      business_logic_plugins: list[dict] # 注入するビジネスロジック
      safety_checks: dict               # 各チェックの合否
      deployment_config: dict            # デプロイ設定

  グラフ:
    parse_spec → extract_features → match_components
        → build_composition_graph → identify_missing
        ↓ (全コンポーネント揃っている)
    inject_business_logic → run_integration_test → deploy_staging
        → user_review → deploy_production
        ↓ (コンポーネントが足りない)
    generate_component → ast_validate → security_scan
        → sandbox_test → human_review → register_to_library
        → (match_componentsに戻る)

■ DB スキーマ

  -- アトミックコンポーネント定義
  CREATE TABLE atomic_components (
    id UUID PRIMARY KEY,
    component_id TEXT UNIQUE NOT NULL,       -- "approval_flow_v2"
    component_name TEXT NOT NULL,            -- "承認ワークフロー"
    category TEXT NOT NULL,                  -- data/flow/calculation/ai
```

```
    version TEXT NOT NULL DEFAULT '1.0.0',
    input_schema JSONB NOT NULL,
    output_schema JSONB NOT NULL,
    config_schema JSONB NOT NULL,
    dependencies TEXT[],                      -- 依存する他コンポーネントID
    code_ref TEXT NOT NULL,                   -- コードリポジトリ参照
    avg_setup_time_hours NUMERIC(4,1),
    reliability_score NUMERIC(3,2),           -- 過去の稼働実績から算出
    usage_count INTEGER DEFAULT 0,
    is_generated BOOLEAN DEFAULT false,       -- AI生成か手動作成か
    safety_check_passed BOOLEAN DEFAULT false,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
  );

  -- システムコンポジション（コンポーネントの組み合わせ定義）
  CREATE TABLE system_compositions (
    id UUID PRIMARY KEY,
    company_id UUID REFERENCES companies(id),
    crystallization_id UUID REFERENCES system_crystallization_points(id),
    system_name TEXT NOT NULL,
    composition_graph JSONB NOT NULL,         -- ノード(コンポーネント)+エッジ(データフロー)
    component_count INTEGER NOT NULL,
    business_logic_plugins JSONB DEFAULT '[]',
    estimated_setup_days NUMERIC(4,1),
    status TEXT DEFAULT 'designing',          --
designing/assembling/testing/staging/production
    version TEXT DEFAULT '1.0.0',
    deployment_url TEXT,
    replaced_saas TEXT[],
    monthly_saving INTEGER,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
  );

  -- コンポーネントインスタンス（個社デプロイ済みの実体）
  CREATE TABLE component_instances (
    id UUID PRIMARY KEY,
    composition_id UUID REFERENCES system_compositions(id),
    component_id UUID REFERENCES atomic_components(id),
    instance_config JSONB NOT NULL,           -- コンポーネント固有の設定値
    business_logic_plugin JSONB,              -- 注入されたビジネスロジック
    source_knowledge_ids UUID[],              -- プラグインの元になったナレッジ
    status TEXT DEFAULT 'active',
    error_count INTEGER DEFAULT 0,
    last_error_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
  );
```

4.3 SaaS Migration Safety Engine — ゼロダウンタイム移行

課題: いきなりSaaSを止めると業務が止まる。「3段階移行」だけでは具体性が不足。データの移行方法、並行稼働時の整合性保証、自動ロールバック条件、APIのないSaaSからのデータ抽出方法が未設計。

なぜ重要か: SaaS移行の失敗は即座に業務停止を意味する。請求書が発行できない、勤怠が記録できない、受注データにアクセスできない — これらは中小企業にとって致命的。シャチョツーが信頼を失う最大のリスクは「移行で業務が止まった」という事態。航空機の飛行中エンジン交換のように、業務を1秒も止めずに移行を完了する安全エンジンが必須。

■ 提案: 5層移行パイプライン + シャドウモード + カナリアデプロイ + 自動ロールバック

[Layer 1] データマイグレーションパイプライン

SaaSのデータをシャチョツーに移行するパイプライン。

Step 1: スキーママッピング（SaaSフィールド → シャチョツーフィールド）

自動マッピング:
・SaaS APIのスキーマ定義を取得
・シャチョツーのターゲットスキーマとフィールド名をEmbedding比較
・類似度 ≥ 0.85 → 自動マッピング
・類似度 0.5–0.85 → 候補表示 → 社長に確認
・類似度 < 0.5 → 新規フィールド作成を提案

例: kintone → シャチョツー
kintone「会社名」 → target「client_name」 (自動: 0.92)
kintone「金額」 → target「amount」 (自動: 0.95)
kintone「備考」 → target「notes」 (自動: 0.88)
kintone「status」 → target「order_status」 (候補: 0.72 → 確認)
kintone「f_001」 → ??? (不明: 0.31 → 新規提案)

Step 2: 変換ルール定義

フィールドの型変換・値変換ルール:
・日付形式: "2024/01/15" → "2024-01-15T00:00:00+09:00"
・金額: 文字列"1,500,000" → integer 1500000
・ステータス: "受注" → "confirmed", "見込" → "prospect"
・計算フィールド: kintoneの自動計算 → シャチョツーの計算式に変換
・関連レコード: kintoneのルックアップ → シャチョツーのリレーション

Step 3: バリデーション + インクリメンタル同期

初回: 全件同期（フルダンプ → 変換 → バリデーション → インポート）
以降: 差分同期（更新タイムスタンプベース → 差分のみ同期）

バリデーションルール:
・件数一致チェック: SaaS件数 = シャチョツー件数
・合計値チェック: SaaS金額合計 = シャチョツー金額合計
・サンプル照合: ランダム10%のレコードを完全照合
・参照整合性: 外部キーの参照先が全て存在するか

[Layer 2] シャドウモード（並行稼働 + 出力比較）

旧SaaSとシャチョツーの新システムを並行稼働させ、出力を自動比較する。

動作イメージ:
1. BPO Workerが通常通り旧SaaSに操作を実行（本番）
2. 同時に、シャチョツーの新システムにも同じ操作を実行（シャドウ）

```
    3. 両方の出力を比較 → 乖離を記録

  乖離率の計算:
    divergence_rate = 不一致件数 / 全比較件数

    比較対象:
      ・データ値の一致（金額・日付・テキスト）
      ・計算結果の一致（集計値・帳票の数値）
      ・ステータスの一致（承認状態・処理状態）
      ・タイミングの一致（処理完了時刻の差 ≤ 5分）

  シャドウモードの段階:
    Week 1-2: 読み取り系のみシャドウ（GET操作 → 結果比較）
      → 目標乖離率: ≤ 5%
    Week 3-4: 書き込み系もシャドウ（POST操作 → 下書き状態で比較）
      → 目標乖離率: ≤ 2%
    Week 5+:  全操作シャドウ
      → 目標乖離率: ≤ 1% を2週間維持でカナリアに進む

  乖離が検出された場合:
    ・乖離パターンを分類（型変換ミス/計算ロジック差/タイミング差/データ欠損）
    ・自動修正可能 → 変換ルールを更新 → 次回から修正
    ・自動修正不可 → 社長に報告 + 開発チームに修正チケット

[Layer 3] カナリアデプロイ（段階的トラフィック移行）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

  シャドウモード通過後、実際のトラフィックを段階的に新システムに移行。

  カナリア段階:
    Stage 1: 10% — 特定の低リスク操作のみ新システムで実行
      ・読み取り系（データ表示・レポート生成）を新システムで
      ・書き込みは引き続き旧SaaS
      ・期間: 1週間 / 昇格条件: エラー率 ≤ 0.5%

    Stage 2: 30% — 下書き・計算系も新システムで実行
      ・下書き作成・集計計算を新システムで
      ・確定操作は引き続き旧SaaS
      ・期間: 1週間 / 昇格条件: エラー率 ≤ 0.3%

    Stage 3: 70% — 確定操作の大部分を新システムで実行
      ・旧SaaSは高リスク操作（外部送信・法令関連）のみ
      ・期間: 1週間 / 昇格条件: エラー率 ≤ 0.1%

    Stage 4: 100% — 完全移行
      ・旧SaaSは読み取り専用（30日間保持後解約）
      ・移行完了レポート自動生成

[Layer 4] 自動ロールバックエンジン
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

  「元に戻すボタン」ではなく、異常を自動検出して自動ロールバック。
```

```
    自動ロールバックトリガー:
      ① エラー率急上昇:
          直近1時間のエラー率 > 5% (基準値の10倍)
          → 即座に旧SaaSに全トラフィックを戻す

      ② レスポンス劣化:
          直近30分の平均レスポンス > 3秒 (基準値の3倍)
          → 新システムのヘルスチェック → 改善なければロールバック

      ③ データ整合性異常:
          定期整合性チェック (5分間隔) で不一致検出
          → 不一致件数 > 3 → 即ロールバック + 不一致データの詳細レポート

      ④ ユーザー報告:
          社長/ユーザーから「おかしい」報告
          → 1クリックで即ロールバック (管理画面に緊急ボタン)

    ロールバック実行:
      1. 新システムへのトラフィックを停止 (0.5秒以内)
      2. 旧SaaSへのトラフィックを再開
      3. 新システムで作成されたデータを旧SaaSに逆同期
      4. ロールバック原因を自動分析 → レポート生成
      5. 修正後、前のカナリアステージから再開

  [Layer 5] SaaS APIリバースエンジニアリング (エクスポートAPI非対応SaaS向け)
  ═══════════════════════════════════════════════════════════

    エクスポートAPIがないSaaSからのデータ抽出方法。

    手法1: UI操作 + スクレイピング (Computer Use活用)
      ・Computer Use で管理画面にログイン
      ・一覧画面をスクロール → スクリーンショット → OCR → データ抽出
      ・CSV出力機能がある場合はCSVダウンロード操作を自動化

    手法2: API観察 (ブラウザのNetwork通信をキャプチャ)
      ・Chrome拡張でSaaS操作時のAPIリクエストをキャプチャ
      ・リクエスト/レスポンスのパターンから非公開APIのスキーマを推定
      ・推定APIを使って直接データ取得 (非公式のため、変更リスク高)
      ・⚠️ 利用規約チェック必須 → 違反の可能性がある場合は手法1に切替

    手法3: 手動エクスポート支援
      ・SaaSの画面上で「エクスポート」ボタンの位置を検出
      ・ユーザーに「この画面でCSV出力してアップロードしてください」とガイド
      ・アップロードされたCSVを自動パース → 移行パイプラインに投入

■ LangGraph 状態遷移

  class MigrationState(TypedDict):
      company_id: str
      source_saas: str                    # 移行元SaaS名
      target_system_id: str               # 移行先システムID
```

```
      schema_mapping: dict                # フィールドマッピング
      transformation_rules: list[dict]    # 変換ルール
      sync_status: dict                   # 同期状況（件数・整合性）
      shadow_mode: dict                   # シャドウモードの状態
      divergence_log: list[dict]          # 乖離ログ
      canary_stage: int                   # 現在のカナリアステージ（0-4）
      health_metrics: dict                # エラー率・レスポンス・整合性
      rollback_triggered: bool
      rollback_reason: str | None

  グラフ:
    analyze_source_schema → create_mapping → define_transformations
        → initial_sync → validate_data → start_shadow_mode
        ↓
    shadow_compare → check_divergence
        ↓ (乖離率OK)               ↓ (乖離率NG)
    canary_stage_1              fix_transformation → re_sync
        → monitor_health
        ↓ (正常)                   ↓ (異常)
    canary_stage_2              auto_rollback → analyze_failure
        → ... → canary_stage_4
        → decommission_saas → generate_report

■ DB スキーマ

  -- SaaS移行プロジェクト
  CREATE TABLE saas_migrations (
    id UUID PRIMARY KEY,
    company_id UUID REFERENCES companies(id),
    composition_id UUID REFERENCES system_compositions(id),
    source_saas TEXT NOT NULL,
    source_saas_config JSONB,               -- API URL, 認証情報参照
    schema_mapping JSONB NOT NULL,          -- {source_field: target_field, ...}
    transformation_rules JSONB NOT NULL,    -- 変換ルール定義
    sync_mode TEXT DEFAULT 'initial',       --
initial/incremental/shadow/canary/complete
    canary_stage INTEGER DEFAULT 0,         -- 0:未開始 / 1-4:カナリア段階
    canary_traffic_percent INTEGER DEFAULT 0,
    last_sync_at TIMESTAMPTZ,
    record_count_source INTEGER,
    record_count_target INTEGER,
    data_integrity_score NUMERIC(5,4),      -- 整合性スコア (0.0-1.0)
    status TEXT DEFAULT 'planning',         --
planning/syncing/shadow/canary/complete/rolled_back
    rollback_count INTEGER DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
  );

  -- 移行チェックポイント（各段階の結果記録）
  CREATE TABLE migration_checkpoints (
    id UUID PRIMARY KEY,
```

```
    migration_id UUID REFERENCES saas_migrations(id),
    checkpoint_type TEXT NOT NULL,           --
sync_complete/shadow_start/canary_1/canary_2/...
    metrics JSONB NOT NULL,                  -- {record_count, integrity_score,
error_rate, ...}
    passed BOOLEAN NOT NULL,
    failure_details JSONB,
    decided_by UUID REFERENCES users(id),   -- 手動判断の場合
    created_at TIMESTAMPTZ DEFAULT NOW()
  );

  -- シャドウモード比較ログ
  CREATE TABLE shadow_comparison_log (
    id UUID PRIMARY KEY,
    migration_id UUID REFERENCES saas_migrations(id),
    operation_type TEXT NOT NULL,            -- read/create/update/delete
    source_result JSONB NOT NULL,            -- 旧SaaSの結果
    target_result JSONB NOT NULL,            -- 新システムの結果
    is_match BOOLEAN NOT NULL,
    divergence_type TEXT,                    --
type_mismatch/value_mismatch/timing/missing
    divergence_detail JSONB,
    auto_fixed BOOLEAN DEFAULT false,
    fix_applied JSONB,                       -- 適用した修正ルール
    created_at TIMESTAMPTZ DEFAULT NOW()
  );

  -- ロールバックイベントログ
  CREATE TABLE migration_rollback_log (
    id UUID PRIMARY KEY,
    migration_id UUID REFERENCES saas_migrations(id),
    trigger_type TEXT NOT NULL,              --
error_rate/response_time/integrity/user_report
    trigger_detail JSONB NOT NULL,
    from_canary_stage INTEGER,
    to_canary_stage INTEGER,                -- ロールバック先のステージ
    data_reverse_synced INTEGER,            -- 逆同期したレコード数
    downtime_seconds INTEGER DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW()
  );
```

4.4 System Evolution Engine — 自律的システム改善

課題: システムは「作って終わり」ではない。SIerの世界では仕様変更のたびに追加見積もりが発生し、システムは陳腐化していく。シャチョツーが作るシステムも、ナレッジの変化・使用パターンの変化・外部環境の変化に自動で適応しなければ、結局SIerと同じになる。

なぜ重要か: シャチョツーの最大の差別化は「変更が即時」。ナレッジが変われば → ビジネスロジックプラグインが自動更新 → システムが自動適応。これがSIerの「仕様変更=追加見積もり2週間」を完全に過去のものにする。さらに、使用データを分析して「この機能は使われていない」「この処理が遅い」を自動検出し、改善提案まで行う自律進化エンジンが、SIer保守契約（月50万）を不要にする。

■ 提案: 4つの自律改善メカニズム

[1] Usage Analytics → 自動改善提案

全操作をトラッキングし、改善ポイントを自動検出する。

トラッキング対象:
・ページビュー: どの画面がどの頻度で使われているか
・操作パス: ユーザーが画面をどう遷移しているか
・滞在時間: 各画面での滞在時間（長い = 分かりにくい？）
・入力パターン: どのフィールドがいつも同じ値で入力されるか
・エラー発生: どの操作でバリデーションエラーが起きるか
・未使用機能: 作ったが1ヶ月以上使われていない機能

自動検出パターンと改善提案:

Pattern A: 同じ値の繰り返し入力
検出: field_X の値が直近50回のうち80%以上同じ
提案: 「この項目のデフォルト値を "○○" に設定しますか？」
効果: 入力時間 -3秒/回 × 50回/月 = 月2.5分削減

Pattern B: 未使用機能の検出
検出: 特定のコンポーネントが30日間使用回数ゼロ
提案: 「"○○レポート" 機能が1ヶ月使われていません。
非表示にするか、削除しますか？」
効果: UI簡素化 → 他機能の使いやすさ向上

Pattern C: ボトルネック画面の検出
検出: 特定の画面の平均滞在時間が他画面の3倍以上
提案: 「"見積もり入力" 画面の操作に平均5分かかっています。
入力項目を減らすか、自動入力を追加しますか？」
改善: LLMで入力フィールドの自動補完を追加

Pattern D: エラー頻発フィールド
検出: 特定のフィールドでバリデーションエラーが月10回以上
提案: 「"税率" フィールドでエラーが頻発しています。
選択式に変更するか、自動設定にしますか？」
改善: テキスト入力 → ドロップダウン or 自動計算に変更

[2] A/B Testing for Business Logic

ナレッジが変更された時、新旧ロジックをA/Bテストで比較する。

トリガー: ナレッジの更新イベント → ビジネスロジックプラグインの変更検出

実行フロー:
1. 旧プラグイン（Variant A）と新プラグイン（Variant B）を並行稼働
2. トラフィックを50:50で分割（同一ユーザーは同じVariantに固定）
3. 以下のメトリクスを比較:

```
        ・処理時間
        ・エラー率
        ・承認時の修正率（少ない方が正確）
        ・ユーザー満足度（👍/👎）
    4. 2週間後 or サンプル数50以上で自動判定:
        ・Variant Bが統計的に有意に優れている → 全トラフィックをBに
        ・有意差なし or Aが優れている → Aを維持 + ナレッジ変更を再検証

  例:
    ナレッジ変更: 「諸経費率を25%→22%に変更」
    A: 旧ルール（25%）で見積もり作成
    B: 新ルール（22%）で見積もり作成
    結果: Bの方が見積もり承認率が15%高い → Bを採用

  安全弁:
    ・金額影響が100万円超の場合は自動A/B禁止 → 社長承認が必要
    ・法令関連のロジック変更は自動A/B禁止

[3] Self-Healing（自動修復）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

  よくある障害を自動検出・自動修復する。SIerの保守契約を不要にする。

  自動修復パターン:

    ① スキーマドリフト修復
      検出: 外部SaaSのAPIレスポンススキーマが変更された
      修復: 新旧スキーマをLLMで比較 → フィールドマッピングを自動更新
      → 修復できた場合: ログ記録 + 社長に「更新しました」通知
      → 修復できない場合: データ同期を一時停止 + 開発チームに通知

    ② APIバージョン変更対応
      検出: APIリクエストが4xx/5xxを返し始めた
      修復:
        Step 1: APIドキュメントをWebFetchで取得
        Step 2: 変更点をLLMが分析
        Step 3: リクエスト形式を自動修正
        Step 4: テスト実行 → 成功なら適用
      → 月間で平均2-3回発生。SIerなら対応に1-3日。シャチョツーは5分。

    ③ パフォーマンス劣化の自動修復
      検出: 特定のクエリのレスポンス時間が前週比200%以上
      修復:
        Step 1: Supabaseのクエリプランを分析
        Step 2: インデックス不足を検出 → 自動CREATE INDEX
        Step 3: N+1クエリを検出 → JOINに書き換え
      → レスポンス改善後、変更内容をログ記録

    ④ ディスク/行数アラート
      検出: テーブルの行数が閾値超過 or Supabaseのストレージ使用率80%超
      修復: 古いログデータのアーカイブ提案 + 自動アーカイブ実行
```

```
  [4] ナレッジ連動バージョニング
  ━━━
```

```
  -- システム進化ログ（全変更を追跡）
  CREATE TABLE system_evolution_log (
    id UUID PRIMARY KEY,
    composition_id UUID REFERENCES system_compositions(id),
    old_version TEXT NOT NULL,
    new_version TEXT NOT NULL,
    change_type TEXT NOT NULL,                --
knowledge_sync/auto_fix/ab_test/user_request/self_heal
    change_detail JSONB NOT NULL,             -- 変更の詳細
    trigger_knowledge_id UUID,               -- トリガーとなったナレッジ変更
    test_results JSONB,                       -- テスト結果
    rollback_available BOOLEAN DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT NOW()
  );

  -- A/Bテスト管理
  CREATE TABLE system_ab_tests (
    id UUID PRIMARY KEY,
    composition_id UUID REFERENCES system_compositions(id),
    component_id UUID REFERENCES atomic_components(id),
    variant_a_config JSONB NOT NULL,          -- 旧ロジック
    variant_b_config JSONB NOT NULL,          -- 新ロジック
    trigger_knowledge_id UUID,               -- トリガーとなったナレッジ変更
    traffic_split NUMERIC(3,2) DEFAULT 0.50, -- Variant Bへのトラフィック割合
    sample_size_target INTEGER DEFAULT 50,
    current_sample_size INTEGER DEFAULT 0,
    metrics JSONB DEFAULT '{"a": {}, "b": {}}',  -- 各バリアントの計測値
    winner TEXT,                              -- a/b/inconclusive
    status TEXT DEFAULT 'running',            -- running/completed/cancelled
    started_at TIMESTAMPTZ DEFAULT NOW(),
    completed_at TIMESTAMPTZ
  );

  -- 自動修復ログ
  CREATE TABLE self_healing_log (
    id UUID PRIMARY KEY,
    composition_id UUID REFERENCES system_compositions(id),
    issue_type TEXT NOT NULL,                 --
schema_drift/api_version/performance/storage
    issue_detail JSONB NOT NULL,
    healing_action JSONB NOT NULL,            -- 実行した修復アクション
    success BOOLEAN NOT NULL,
    manual_intervention_needed BOOLEAN DEFAULT false,
    resolved_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
  );
```

4.5 System Pattern Network Intelligence — 全社横断パターン学習

課題: 業種Aで作ったカスタムシステムが業種Bでも使えるケースが多い。単純なテンプレートの横展開では「業種ごとの微妙な違い」に対応できない。さらに、「日報+在庫管理=現場管理統合」のようなパターンの組み合わせ、類似企業への先回り推薦ができていない。

なぜ重要か: 100社に展開した時点で、各社が個別にシステムを作っていたら100通りの保守が必要。しかし実際は80%が共通パターンの組み合わせ。パターンを抽象化・組み合わせ・業界適応することで、新規企業には「あなたの会社に必要なシステムはおそらくこの3つです。類似企業50社で実証済み」と即提案できる。これが「SIerに要件定義を依頼する」必要性を完全に消す。

```
■ 提案: パターン抽象化 + コンポジション代数 + 業界適応 + 予測推薦

  [1] Pattern Abstraction Engine（成功システムからのパターン抽出）
  ═══════════════════════════════════════════════════════

    稼働中のシステムから、個社固有の情報を除去してパターンを抽出する。

    抽出プロセス:
      1. 稼働中システムのコンポジショングラフを取得
      2. ビジネスロジックプラグインを「プレースホルダー」に置換
         例: {formula: "cost * 0.25"} → {formula: "cost * {overhead_rate}"}
      3. 個社データ（会社名・具体的金額・担当者名）を除去
      4. コンポーネント構成 + データフロー + プレースホルダーロジック = パターン
      5. パターンのハッシュを計算 → 既存パターンとの類似度チェック
         類似度 ≥ 0.9 → 既存パターンのバリエーションとして登録
         類似度 < 0.9 → 新規パターンとして登録

    例: 建設会社A → 「受注→請求一体化パターン」を抽出
      コンポーネント: data_input → calculation_engine → approval_flow
                      → pdf_builder → notification
      プレースホルダー:
        ・{overhead_rate}: 諸経費率（企業ごとに15-30%で異なる）
        ・{approval_stages}: 承認段数（1-3段で異なる）
        ・{invoice_template}: 請求書テンプレート（企業ロゴ等）
        ・{notification_channel}: 通知先（LINE/Slack/メール）

  [2] Pattern Composition Algebra（パターンの組み合わせ）
  ═══════════════════════════════════════════════════════

    パターン同士を「合成」して新しい統合パターンを作る。

    合成の種類:
      ① 直列合成（Sequential）: A → B
         例: 「受注管理」→「請求管理」= 「受注→請求一体化」
         条件: Aのoutput_schemaとBのinput_schemaに共通フィールドが存在

      ② 並列合成（Parallel）: A || B
         例: 「日報管理」||「在庫管理」= 「現場管理統合」
         条件: AとBが同じデータソースを共有（data_input コンポーネント共有）

      ③ 拡張合成（Extension）: A + Δ
         例: 「日報管理」+ Δ(写真+天候) = 「建設日報管理」
         条件: Δのフィールドを既存コンポーネントに追加可能

    互換性スコア:
      compatibility(pattern_A, pattern_B) =
        0.40 × schema_overlap        -- データスキーマの重複度
      + 0.30 × component_reuse       -- 共有可能なコンポーネント比率
      + 0.20 × workflow_alignment    -- ワークフローの整合性
      + 0.10 × user_overlap          -- 同じユーザーが使う確率
```

```
    compatibility ≥ 0.7 → 合成推薦対象

  合成の具体例:

    ┌──────────────────────────────────────────────────┐
    │  パターン合成提案                                  │
    ├──────────────────────────────────────────────────┤
    │                                                  │
    │   「日報管理」(利用: 45社) + 「在庫管理」(利用: 38社)  │
    │   → 「現場管理統合」(互換性: 0.82)                  │
    │                                                  │
    │  合成メリット:                                    │
    │     ・日報入力時に在庫使用量も同時記録(二重入力廃止)   │
    │     ・在庫アラートが日報の作業予定と連動              │
    │     ・コンポーネント共有: data_input(1個削減)        │
    │     ・推定効果: 入力時間 -40% / データ整合性向上       │
    │                                                  │
    │  既に利用中の企業: 12社(満足度: 4.6/5.0)            │
    └──────────────────────────────────────────────────┘

[3] Industry-Specific Adaptation Layer(業界適応レイヤー)
═══════════════════════════════════════════════════

  同じベースパターンに対して、業界ごとの「差分(diff)」を管理する。

  ベースパターン(業界共通):
    「日報管理」
      ・日付入力
      ・作業内容テキスト
      ・作業時間
      ・担当者

  業界適応diff:
    建設業:
      + 現場名(必須)
      + 天候(選択式: 晴/曇/雨/雪)
      + 現場写真(複数枚)
      + KY活動記録(安全管理用)
      + 使用資材(在庫連動)

    製造業:
      + ライン名(必須)
      + ロット番号
      + 品質チェック項目(チェックリスト)
      + 不良品数
      + 設備稼働時間

    飲食業:
      + 店舗名
      + 売上(ランチ/ディナー分割)
      + 客数
      + 食材使用量(在庫連動)
      + 衛生チェック項目
```

```
    歯科:
      + 患者数（初診/再診）
      + 自費率
      + キャンセル数
      + 使用材料（在庫連動）

  diffの適用:
    新規企業(製造業)が「日報管理」を導入する場合:
      1. ベースパターンを取得
      2. 「製造業diff」を適用
      3. 個社固有のカスタマイズ（プレースホルダー値を設定）
      4. 即座にシステム生成 → 推定3日で稼働開始

[4] Pattern Quality Scoring（パターン品質の定量評価）
═══════════════════════════════════════════════════════

  パターンの品質を継続的に測定し、低品質パターンを改善する。

  quality_score =
    0.25 × adoption_rate          -- 提案された企業のうち採用した割合
  + 0.25 × satisfaction_score     -- 利用企業の平均満足度（1-5 → 0-1正規化）
  + 0.20 × (1 - maintenance_cost) -- 月間メンテナンス工数の逆数
  + 0.15 × (1 - customization_rate) -- カスタマイズ不要なほど高品質
  + 0.15 × stability_score        -- エラー率の低さ × 稼働月数

  品質ランク:
    proven   (0.8-1.0): 50社以上で実績。推薦時に最優先
    stable   (0.6-0.8): 10社以上で安定稼働。標準推薦
    beta     (0.3-0.6): 3-10社で検証中。「ベータ版」と明記して推薦
    draft    (0.0-0.3): 1-2社のみ。推薦対象外。改善パイプラインへ

  改善パイプライン:
    ・beta以下のパターンに対して:
      1. カスタマイズ率30%以上のフィールドを特定
      2. 「このフィールドをパターンのデフォルトオプションに追加すべきか」をLLMが判定
      3. 追加 → パターンバージョンアップ → 既存利用企業に通知

[5] Predictive Pattern Recommendation（予測推薦エンジン）
═══════════════════════════════════════════════════════

  企業のゲノム・BPOパターン・類似企業データから、
  3-6ヶ月以内に必要になるシステムを先回りで推薦する。

  推薦アルゴリズム:
    1. 対象企業のゲノム（遺伝子セット）を取得
    2. 遺伝子の Jaccard係数 ≥ 0.6 の類似企業群を特定
    3. 類似企業群で「導入月の中央値」を計算:
       例: 類似企業50社のうち、導入6ヶ月目に「在庫管理」を
           追加導入した企業が72%
    4. 対象企業の導入月数と照合:
       → 現在4ヶ月目 → 2ヶ月後に「在庫管理」を必要とする確率: 72%
```

```
      5. 確率 ≥ 60% のパターンを推薦

    推薦の表示:
      ┌──────────────────────────────────────────────────────┐
      │  先読み提案 — 3ヶ月以内に必要になりそうなシステム       │
      ├──────────────────────────────────────────────────────┤
      │                                                      │
      │  #1 在庫管理システム                                   │
      │      予測確率: 72% （類似企業50社中36社が導入）          │
      │      導入効果: 月間¥45,000削減（棚卸時間+在庫切れ損失）  │
      │      開発期間: 5日（provenパターン使用）                 │
      │      [今すぐ検討] [3ヶ月後にリマインド]                  │
      │                                                      │
      │  #2 安全書類管理システム                                 │
      │      予測確率: 65% （類似企業50社中33社が導入）          │
      │      導入効果: 月間¥30,000削減（書類作成時間）           │
      │      開発期間: 4日（provenパターン使用）                 │
      │      [今すぐ検討] [3ヶ月後にリマインド]                  │
      └──────────────────────────────────────────────────────┘

■ LangGraph 状態遷移

  class PatternNetworkState(TypedDict):
      pattern_id: str | None
      company_id: str
      genome: dict                          # 企業のゲノム情報
      active_systems: list[dict]            # 稼働中システム一覧
      similar_companies: list[dict]         # 類似企業群
      abstracted_patterns: list[dict]       # 抽出されたパターン
      composition_candidates: list[dict]    # 合成候補
      industry_diffs: dict                  # 業界適応diff
      quality_scores: dict                  # パターン品質スコア
      predictions: list[dict]              # 予測推薦結果

  グラフ:
    abstract_from_systems → register_pattern → calculate_quality
        ↓
    find_compositions → score_compatibility → propose_compositions
        ↓
    find_similar_companies → predict_needs → rank_recommendations
        → present_to_user
        ↓ (新システム完成時)
    extract_new_pattern → merge_with_existing → update_quality_scores

■ DB スキーマ

  -- システムパターン（抽象化されたテンプレート）
  CREATE TABLE system_patterns (
    id UUID PRIMARY KEY,
    pattern_name TEXT NOT NULL,              -- 「受注→請求一体化」「日報管理」等
    pattern_hash TEXT UNIQUE NOT NULL,       -- コンポジション構造のハッシュ
    composition_template JSONB NOT NULL,     -- プレースホルダー付きコンポジショングラフ
```

```
  placeholder_schema JSONB NOT NULL,       -- プレースホルダーの定義と説明
  component_ids UUID[] NOT NULL,           -- 使用するコンポーネントID群
  quality_score NUMERIC(4,3) DEFAULT 0.30,
  quality_rank TEXT DEFAULT 'draft',       -- draft/beta/stable/proven
  adoption_count INTEGER DEFAULT 0,        -- 採用企業数
  proposal_count INTEGER DEFAULT 0,        -- 提案した企業数
  avg_satisfaction NUMERIC(3,2),
  avg_customization_rate NUMERIC(3,2),
  avg_dev_days NUMERIC(4,1),
  avg_monthly_saving INTEGER,              -- 平均月額削減効果
  applicable_genes TEXT[],                 -- 関連する遺伝子ID
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- パターン合成定義
CREATE TABLE pattern_compositions (
  id UUID PRIMARY KEY,
  composed_pattern_id UUID REFERENCES system_patterns(id),  -- 合成結果パターン
  source_pattern_ids UUID[] NOT NULL,      -- 合成元パターンID群
  composition_type TEXT NOT NULL,          -- sequential/parallel/extension
  compatibility_score NUMERIC(4,3) NOT NULL,
  shared_components UUID[],               -- 共有されるコンポーネント
  merge_benefits JSONB,                    -- 合成による効果の説明
  adoption_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 業界適応diff
CREATE TABLE pattern_adaptations (
  id UUID PRIMARY KEY,
  pattern_id UUID REFERENCES system_patterns(id),
  industry TEXT NOT NULL,                  -- 建設/製造/飲食/歯科/...
  adaptation_diff JSONB NOT NULL,          -- 追加フィールド・ルールの差分
  validation_rules JSONB,                  -- 業界固有のバリデーション
  adoption_count INTEGER DEFAULT 0,
  avg_satisfaction NUMERIC(3,2),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(pattern_id, industry)
);

-- 予測推薦ログ
CREATE TABLE pattern_predictions (
  id UUID PRIMARY KEY,
  company_id UUID REFERENCES companies(id),
  pattern_id UUID REFERENCES system_patterns(id),
  prediction_probability NUMERIC(4,3) NOT NULL,  -- 予測確率
  similar_company_count INTEGER NOT NULL,         -- 類似企業数
  similar_adopted_count INTEGER NOT NULL,         -- 類似企業で採用した数
  predicted_need_month INTEGER,                   -- 何ヶ月後に必要と予測
  estimated_monthly_saving INTEGER,
  estimated_dev_days NUMERIC(4,1),
```

```
    user_decision TEXT,                              -- accepted/deferred/rejected
    actual_adopted BOOLEAN,                          -- 実際に採用されたか（予測精度追跡）
    created_at TIMESTAMPTZ DEFAULT NOW()
  );
```

4.5 ブラウザ拡張による透明移行エンジン — 社員が気づかない移行

課題: 現設計はシステムを作って「移行してください」で終わり。実際のボトルネックは社員の反発と再教育コスト。「新しいシステムです、使い方覚えてください」は必ず失敗する。

なぜ重要か: 移行コストがゼロなら、SaaS置換の意思決定が劇的に楽になる。社員が気づかないうちに移行が完了し、ある日「kintone解約しました」と社長が言えばいい。

■ 3フェーズ移行設計

```
Phase A: 並行稼働（2〜4週間）
  - 自社システムとkintoneに同時書き込み
  - データ同期差分率を毎時計測（目標: diff_rate < 0.1%）
  - 社員は引き続きkintoneを使う → 一切の変化なし

Phase B: ブラウザ拡張によるUX再構築（2〜6週間）
  - Chrome拡張をサイレントデプロイ（管理者がGoogle管理コンソールから配布）
  - kintone.com へのアクセスをインターセプト → 自社UXを注入 → 自社APIに書き込み
  - 単なるUI差し替えではなく、業務フロー自体を再設計:
    ・複数画面遷移の1画面統合（例: 3画面で行っていた日報入力を1画面に）
    ・承認フローの自動化（社長の判断基準ベースで自動承認/エスカレーション）
    ・入力順序の最適化（業務フローに沿った自然な順序に並び替え）
    ・不要フィールドの非表示 + 必須フィールドの追加
    ・コンテキスト連動（前工程のデータを自動プリフィル）
    ・リアルタイムバリデーション（社長ルールベースの即時チェック）
  - 社員の体験: 「画面が同じ」ではなく「仕事が楽になった」
  - staff_error_rate + task_completion_time を連続モニタリング
    （error_rate 0.5%以上増加 or completion_time 20%以上悪化で自動ロールバック）

Phase C: 完全移行・解約
  - diff_rate < 0.1% かつ staff_error_rate に有意な変化なし → cutover_ready = true
  - kintoneへの書き込みを停止 → 読み取りのみに切り替え → 30日後に解約手続き
```

■ Computer Use による UI 自動解析・コンポーネント生成

```
async def analyze_and_clone_ui(saas_url: str, company_id: str) -> UICloneResult:
    """
    既存SaaSの画面をComputer Useで解析し、
    shadcn/uiコンポーネントを自動生成する。
    """
    # Computer Useで画面をスクリーンショット → UI要素を抽出
    screenshot = await computer_use.capture(saas_url)
    ui_elements = await llm.analyze_ui_structure(screenshot)

    # フォーム・テーブル・ボタン・バリデーションルールを抽出
    components = []
    for element in ui_elements.form_fields:
        component = await llm.generate_shadcn_component(
            element=element,
            style_guide="match_original_layout",  # レイアウトを極力再現
            api_endpoint=f"/api/v1/{element.resource_name}"
        )
        components.append(component)

    # 差分テスト: 同じ操作をkintoneと自社システムで実行 → 結果比較
    diff_rate = await run_parallel_operations(saas_url, components)

    return UICloneResult(
        components=components,
        diff_rate=diff_rate,
        ready_for_phase_b=(diff_rate < 0.001)  # 0.1%未満
    )
```

■ LangGraph MigrationState

```
class MigrationState(TypedDict):
    company_id: str
    saas_name: str                    # "kintone", "freee" 等
    phase: Literal["A", "B", "C"]
    sync_diff_rate: float             # 同期差分率（目標: < 0.001）
    staff_error_rate: float           # 社員エラー率（ベースラインからの変化）
    browser_extension_active: bool    # 拡張機能デプロイ済み
    extension_install_count: int      # インストール済み端末数
    total_target_devices: int         # 対象端末総数
    cutover_ready: bool               # 完全移行可能フラグ
    rollback_triggered: bool          # 自動ロールバック発動フラグ
    switched_at: Optional[datetime]   # 完全移行完了日時
```

■ DB スキーマ

```
CREATE TABLE ui_migration_log (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id            UUID NOT NULL REFERENCES companies(id),
  saas_name             TEXT NOT NULL,                    -- "kintone", "freee" 等
  phase                 TEXT NOT NULL CHECK (phase IN ('A','B','C')),
  sync_diff_rate        NUMERIC(8,6),                     -- 例: 0.000023 (0.0023%)
  staff_error_rate      NUMERIC(8,6),                     -- ベースラインからの差分
  extension_install_count INT DEFAULT 0,
  total_target_devices  INT,
  cutover_ready         BOOLEAN DEFAULT FALSE,
  rollback_triggered    BOOLEAN DEFAULT FALSE,
  switched_at           TIMESTAMPTZ,
  recorded_at           TIMESTAMPTZ DEFAULT NOW()
);
```

4.6 ナレッジ変更 → システム自動保守パイプライン — 作って終わりにしない

課題: 現設計はシステムを「作って終わり」。業務ルールが変わったとき誰が直すか設計されていない。中小企業でシステムが陳腐化する最大原因は保守コストと人材不在。

なぜ重要か: ブレイン編でナレッジが更新されるたびに自動でシステムに反映されれば、保守コストがほぼゼロになる。「生きているシステム」として常に最新の業務ルールを体現し続ける。

■ 変更規模分類と自動対応フロー

```
ナレッジ変更トリガー
    ↓
影響システム自動検出（knowledge_item_id → system_ids マッピング）
    ↓
変更規模分類
    ├─ 軽微（表示名・閾値・メッセージ変更）
    │      → 自動パッチ適用 → テスト実行 → 通過で自動デプロイ → LINE通知
    │
    ├─ 中程度（フロー変更・条件分岐追加・API変更）
    │      → Engineer Workerが差分コード生成
    │      → 社長に「このナレッジ変更を反映しました。確認してください」
    │      → 承認後デプロイ
    │
    └─ 大規模（テーブル構造変更・ドメインモデル変更）
           → 要件定義からやり直し（4.1〜4.4フローを再実行）
           → 社長に「大きな変更が必要です。打ち合わせをセットしましょう」
```

■ 具体例

変更前: 「請求書は月末締め、翌月末送付」
変更後: 「請求書は月末締め、翌月5日送付」

```
1. ブレイン編でナレッジ更新（knowledge_item_id: abc-123）
2. 影響システム検出: 請求管理システム（system_id: sys-456）
3. 変更規模分類: 軽微（日付パラメータの変更）
4. 自動パッチ生成:
   - InvoiceScheduler.send_day: 月末最終日 → 翌月5日
5. テスト自動実行（当月末→翌月5日の日付計算テスト）
6. 全テスト通過 → 自動デプロイ
7. 社長LINEに通知:
   「✅ 請求書送付日を翌月5日に自動更新しました（変更元: ナレッジ更新 2026/3/12）」
```

■ セキュリティパッチ自動適用

```
依存ライブラリ脆弱性スキャン（毎日 03:00 実行）
    ↓
CVE データベース照合
    ├─ Critical/High → 即時自動PR生成 → テスト実行 → 通過で自動マージ → Slack通知
    ├─ Medium        → 週次バッチで自動PR → 社長承認後マージ
    └─ Low/Info      → 月次レポートに記載のみ
```

■ LangGraph AutoMaintenanceState

```
class AutoMaintenanceState(TypedDict):
    knowledge_change_id: str
    affected_systems: List[str]           # 影響を受けるsystem_idリスト
    change_classification: Literal[       # 変更規模分類
        "minor",      # 軽微: 自動適用
        "moderate",   # 中程度: 社長確認後
        "major"       # 大規模: 要件定義からやり直し
    ]
    patch_generated: bool
    patch_diff_summary: str               # diff の人間可読サマリー
    test_result: Literal["pass", "fail", "pending"]
    test_pass_rate: float                 # テスト通過率（例: 0.98）
    applied_at: Optional[datetime]
    requires_approval: bool               # 社長承認が必要か
    approved_by: Optional[str]            # 承認者のuser_id
```

■ DB スキーマ

```
CREATE TABLE system_maintenance_log (
  id                        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id                UUID NOT NULL REFERENCES companies(id),
  system_id                 UUID NOT NULL,
  triggered_by_knowledge_id UUID,                          -- 変更トリガーとなったナレッジID
  change_type               TEXT NOT NULL CHECK (
                              change_type IN ('auto','assisted','manual')
                            ),
  diff_summary              TEXT,                          -- 変更内容の人間可読サマリー
  test_pass_rate            NUMERIC(5,4),                  -- 例: 0.9800
  applied_at                TIMESTAMPTZ,
  requires_approval         BOOLEAN DEFAULT FALSE,
  approved_by               UUID REFERENCES users(id),
  created_at                TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE system_health_metrics (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id            UUID NOT NULL REFERENCES companies(id),
  system_id             UUID NOT NULL,
  uptime_rate           NUMERIC(7,6),                      -- 例: 0.999850 (99.985%)
  error_rate            NUMERIC(7,6),                      -- 例: 0.000120
  performance_p95       NUMERIC(10,2),                     -- p95レイテンシ（ms）
  last_security_scan    TIMESTAMPTZ,
  vulnerabilities_found INT DEFAULT 0,                     -- 未解決CVE件数
  recorded_at           TIMESTAMPTZ DEFAULT NOW()
);
```

4.7 SaaS解約ロードマップ自動生成 — 「いつ何を解約するか」を設計する

課題: 現設計はシステムを「作る」まで。「いつ何を解約して、いくら削減できるか」のロードマップが自動生成されない。社長は「なんとなくSaaS代が減った気がする」程度の認識になりがち。

なぜ重要か: 解約ロードマップが可視化されると、社長の投資判断が劇的に変わる。「6ヶ月後に月¥95k削減、年間¥1.14M削減」という具体的な数字があれば、開発投資の意思決定が「コスト」から「投資」に変わる。

■ SaaS依存グラフとトポロジカルソート

```
# SaaS依存グラフ: ノード=SaaS、エッジ=データフロー方向
# 例: kintone → freee（kintoneの売上データがfreeeに連携されている）
#
# トポロジカルソートで置換順序を決定
# → データの「送り元」から先に置換する（依存元から先に置換）
#
# 具体例:
#   kintone ──→ freee ──→ マネーフォワードクラウド
#   この場合: kintone → freee → マネーフォワードの順に置換
#   理由: kintoneを先に置換しないとfreeeへのデータ連携が途絶える

def calculate_replacement_order(saas_dependency_graph: Dict) -> List[str]:
    """
    依存グラフをトポロジカルソートして置換順序を返す。
    依存元（データの送り元）が先に来る。
    """
    return topological_sort(saas_dependency_graph)
```

■ 置換難易度スコア

```
置換難易度スコア（0〜1）=
    API品質        × 0.3   # 公式APIあり=0.1、非公式=0.5、APIなし=0.9
  + データ複雑度   × 0.3   # シンプル=0.1、カスタムフィールド多数=0.7
  + 法的制約       × 0.2   # 電子帳簿保存法対応必須=0.8、なし=0.1
  + 社員依存度     × 0.2   # 全員毎日使用=0.9、月1回=0.2

例:
  kintone:  0.3×0.3 + 0.5×0.3 + 0.1×0.2 + 0.8×0.2 = 0.09+0.15+0.02+0.16 = 0.42（中）
  freee:    0.2×0.3 + 0.3×0.3 + 0.8×0.2 + 0.6×0.2 = 0.06+0.09+0.16+0.12 = 0.43（中）
  Slack:    0.1×0.3 + 0.2×0.3 + 0.0×0.2 + 0.9×0.2 = 0.03+0.06+0.00+0.18 = 0.27（低）
```

■ 解約ロードマップ自動生成（月別）

```
月      SaaS名            月額       難易度   依存順   想定解約月   累積削減額（月）
──────────────────────────────────────────────────────────────────────────────
1ヶ月後  Slack             ¥15,000   0.27    1位     1ヶ月後     ¥15,000/月
2ヶ月後  kintone           ¥30,000   0.42    2位     3ヶ月後     ¥45,000/月
3ヶ月後  Zoom              ¥10,000   0.21    3位     3ヶ月後     ¥55,000/月
4ヶ月後  freee（会計）     ¥20,000   0.43    4位     5ヶ月後     ¥75,000/月
5ヶ月後  マネーフォワード  ¥8,000   0.38    5位     6ヶ月後     ¥83,000/月
6ヶ月後  SmartHR           ¥12,000   0.51    6位     7ヶ月後     ¥95,000/月

→ 6ヶ月後: 月¥95,000削減 = 年間¥1,140,000削減
```

Before/After比較:

項目	移行前	移行後（6ヶ月後）
月額SaaS費	¥95,000/月	¥0/月
年間SaaS費	¥1,140,000/年	¥0/年
システム運用コスト	¥95,000/月（SaaS費のみ）	シャチョツー利用料のみ
保守作業	手動（担当者が対応）	自動（4.6パイプライン）

■ 解約の法的チェック自動カウントダウン

```
kintone 解約予告: 残り 45日
  ├─ 契約満了日: 2026/5/31
  ├─ 解約予告必要日数: 30日前
  ├─ 解約申請期限: 2026/5/1（今日から45日後）
  └─ ⚠️ 2026/4/15 に「解約申請してください」とLINE通知予定

freee 解約予告: まだ移行準備中（3ヶ月後に通知予定）
```

■ LangGraph CancellationRoadmapState

```
class CancellationRoadmapState(TypedDict):
    company_id: str
    saas_dependency_graph: Dict[str, List[str]]   # {saas_name: [依存先saas_nameリスト]}
    replacement_order: List[str]                   # トポロジカルソート結果
    monthly_milestones: List[Dict]                 # 月別マイルストーン（SaaS名・想定解約月・削
減額）
    cumulative_savings_projection: Dict[int, int]  # {month: 累積削減月額} 例: {6: 95000}
    total_annual_savings: int                      # 年間削減額（例: 1140000）
    next_action: str                               # 次に取るべきアクション
    legal_deadlines: List[Dict]                    # 解約期限アラートリスト
```

■ DB スキーマ

```
CREATE TABLE saas_cancellation_roadmap (
  id                        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id                UUID NOT NULL REFERENCES companies(id),
  saas_name                 TEXT NOT NULL,
  monthly_fee               INTEGER NOT NULL,              -- 月額（円）
  replacement_system_id     UUID,                          -- 置換後の自社システムID
  dependency_order          INTEGER,                       -- 置換順序（1始まり）
  replacement_difficulty_score NUMERIC(4,2),               -- 例: 0.42
  contract_end_date         DATE,                          -- 契約満了日
  notice_required_days      INTEGER DEFAULT 30,            -- 解約予告必要日数
  target_cancel_month       DATE,                          -- 想定解約月（YYYY-MM-01）
  actual_cancelled_at       TIMESTAMPTZ,                   -- 実際に解約した日時
  cumulative_savings         INTEGER,                      -- この解約による累積月額削減額
 （円）
  created_at                TIMESTAMPTZ DEFAULT NOW(),
  updated_at                TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE saas_dependency_edges (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id            UUID NOT NULL REFERENCES companies(id),
  from_saas             TEXT NOT NULL,                     -- データの送り元SaaS
  to_saas               TEXT NOT NULL,                     -- データの受け取り先SaaS
  data_flow_description TEXT,                              -- 例: "売上データをfreeeに連携"
  flow_volume_monthly   INTEGER,                           -- 月次データ件数（概算）
  created_at            TIMESTAMPTZ DEFAULT NOW()
);
```

---

5. 「SaaSを潰す」キラーシナリオ

```
■ Before（SaaS + SIerの世界）

  社長: 「うちのITコスト、高くない？」
  → kintone: ¥50,000/月（30ユーザー）
  → freee:   ¥30,000/月
  → ジョブカン: ¥20,000/月
  → Slack:   ¥15,000/月（30名）
  → ANDPAD:  ¥50,000/月
  → SIer保守: ¥500,000/月
  → 合計: ¥665,000/月 = ¥7,980,000/年

  しかも:
  → 各SaaS間の連携はSIerに別途依頼（初期200万 + 月20万保守）
  → データがバラバラでレポート作成に毎月3日かかる
  → 仕様変更のたびにSIerへ追加見積もり

■ After（シャチョツーの世界 — 3年後の姿）

  社長: 「シャチョツーだけでいいんじゃない？」
  → シャチョツー: ¥80,000/月（基本料+従量課金）
     ・ナレッジ管理 + Q&A（旧kintoneの一部を代替）
     ・請求処理 Level 4 自動化（freeeの主要機能を代替）
     ・日報システム内蔵（kintoneの日報機能を代替）
     ・コミュニケーション → LINE/Slack連携（Slackは残す判断も可）
     ・勤怠データ → 専用画面（ジョブカンの基本機能を代替）
     ・建設現場管理 → 写真+日報+天候の統合管理
     ・SIerの仕事 → AIが代替（ヘルプデスク+連携+セキュリティ）

  → 月額: ¥665,000 → ¥80,000 = 年間700万削減
  → しかもデータが一元化されているのでレポートは即生成
  → 仕様変更はナレッジを変えるだけで自動反映

■ 注意: 全SaaSを潰すわけではない

  残すもの:
  ・法令で必須のもの（e-Tax連携のため最低限のfreee等）
  ・取引先指定のもの（元請指定のANDPAD等）
  ・深い専門機能（会計仕訳AIなど、シャチョツーより優れている部分）

  潰すもの:
  ・「データの入れ物」としてしか使っていないSaaS
  ・機能の20%しか使っていないSaaS
  ・連携のためだけに存在するiPaaS
  ・SIerの定型保守（ヘルプデスク+バックアップ+監視）
```

6. Phase対応表

本ドキュメント	02_プロダクト設計.md	03_実装計画.md
Phase 5後半: Engineer Worker	Layer 3b (Engineer Worker)	Phase 5
Phase 6: SaaS置換 + セキュリティ	Layer 3b (完全版) + 15.2 SIer置換	Phase 6
→ ここで「SaaS/SIer不要」を実証	L3 完成	Phase C-D (100-300社)

6.5 SMB / エンプラ 自社システム展開の設計差異

SMBとエンプラで何が変わるか

```
■ SMB（～300名）向け自社システム
  目的: SaaS月額費用の削減・業務最適化
  展開方式: シャチョツー共有インフラ上にデプロイ
  セキュリティ: 00_00_セキュリティ設計.md §1-§17
  開発期間: 1-4週間（アトミックコンポーネント組み合わせ）
  コスト: 初期¥0（月額プランに含む）+ インフラ従量

■ エンプラ（300名～）向け自社システム
  目的: 上記 + 既存基幹システムとの統合・社内セキュリティポリシー準拠
  展開方式: 専有インスタンス or 顧客GCPプロジェクトへのデプロイ
  セキュリティ: 00_00_セキュリティ設計.md §1-§26（全て）
  開発期間: 2-8週間（審査・テスト工程が増える）
  コスト: 専有インスタンス費用 + 開発工数（別途見積）
```

エンプラ向け自社システム固有の追加設計

① 生成コードのセキュリティ審査フロー

```
SMB:
  LLMコード生成 → Semgrep自動スキャン → デプロイ
  （自動スキャンでCritical/High未検出なら即デプロイ）

エンプラ:
  LLMコード生成
    ↓
  Semgrep自動スキャン（§16）
    ↓
  顧客の情報システム部門によるコードレビュー（5営業日）
    ↓
  ステージング環境での受け入れテスト（顧客実施）
    ↓
  顧客の変更管理委員会（CAB）承認
    ↓
  本番デプロイ（メンテナンスウィンドウ内）

  ※ CAB: Change Advisory Board（変更管理委員会）
     大企業では全システム変更にCAB承認が必要

■ 生成コードの提出物（エンプラ標準）：
  ・ソースコード（GitHubリポジトリへのアクセス付与）
  ・セキュリティスキャン結果レポート（Semgrep / Trivy）
  ・アーキテクチャ設計書（データフロー図付き）
  ・テスト仕様書 + テスト結果
  ・運用手順書（障害時・定期メンテナンス）
  ・データ項目一覧（個人情報の取り扱いを含む）
```

② 既存基幹システムとの統合設計

```
エンプラが持つ既存システム（中小企業にはないもの）：
  ・ERP（SAP / Oracle / 勘定奉行クラウド）
  ・グループウェア（Microsoft 365 / Notes）
  ・ID管理（Active Directory / LDAP）
  ・ファイルサーバー（オンプレミス）

統合パターン：

  [パターン1] API統合（ERP等がAPI公開している場合）
    自社システム ←→ ERP API
    データ同期: リアルタイム or 夜間バッチ

  [パターン2] DB直結（許可された場合）
    自社システム → 読み取り専用接続 → ERP DB
    ※ ERPベンダーの承認が必要

  [パターン3] ファイル連携（最も多いケース）
    ERP → CSVエクスポート → GCS → 自社システムが取り込み
    スケジュール: 毎朝6時バッチ処理

  [パターン4] iPaaS経由（Make / Zapier）
    既存システム → Make → 自社システム
    ※ BPO編 4.4のコネクタ戦略と同一設計

■ 統合時の責任分界点
  シャチョツーの責任範囲: 自社システム側の設計・開発・保守
  顧客の責任範囲: 既存システム側のAPI提供・認証情報管理・ファイル出力
  共同責任: データ形式の仕様定義・テスト実施
```

③ エンプラ向けデプロイオプション

```
[Option A] シャチョツー共有インフラ（デフォルト）
  ・Cloud Run（東京リージョン）
  ・Supabase（マルチテナント）
  ・月額: 通常プランに含む
  ・対象: ～300名、機密性要件が標準レベル

[Option B] 専有インスタンス（エンプラ標準）
  ・専用Cloud Run + 専用Supabase
  ・他社データと物理分離
  ・月額: +¥150,000
  ・対象: 300名～、「他社と同じDBは困る」要件がある場合

[Option C] 顧客GCPプロジェクトデプロイ（最高セキュリティ）
  ・顧客が所有するGCPプロジェクトにデプロイ
  ・データが顧客インフラから外に出ない
  ・初期費用: ¥500,000 + 月額保守¥100,000
  ・対象: 金融・医療・防衛・機密性最高レベル

[Option D] オンプレミスデプロイ（将来対応）
  ・Dockerコンテナで顧客データセンターにデプロイ
  ・インターネット接続なしで動作
  ・LLMはオンプレミスモデル（Llama等）に切替
  ・対象: 完全クローズド環境が必要な顧客（国防・原子力等）
```

④ エンプラ向け保守・SLAオプション

```
■ SMBとエンプラの保守の違い

  SMB:
    自動保守パイプライン（§4.6）がナレッジ変更を検知して自動パッチ
    社長への通知のみ

  エンプラ:
    自動パッチは「検証環境へのデプロイ」まで（本番は自動デプロイしない）
    変更管理委員会（CAB）への変更申請書を自動生成
    本番デプロイは定期メンテナンスウィンドウ内（毎週水曜22:00–24:00等）

■ エンプラ向けSLAオプション

  Standard（エンプラ基本）:
    稼働保証: 99.9%/月
    サポート: 平日9–18時（日本語）
    インシデント対応: P0=1時間以内、P1=4時間以内

  Premium（エンプラ上位）:
    稼働保証: 99.95%/月
    サポート: 24時間365日（日本語）
    インシデント対応: P0=15分以内、P1=1時間以内
    専任カスタマーサクセス担当

■ 技術サポート対応範囲
  シャチョツー提供システムの障害対応: ○（範囲内）
  顧客既存システムとの統合部分の障害: △（初期調査のみ、修正は別途費用）
  顧客側インフラの障害: ×（範囲外）
```

エンプラ導入フロー（SMBとの違い）

```
SMB導入（最短1日）:
  LINE登録 → 業種選択 → SaaS接続（1タップ） → 開始

エンプラ導入（標準8週間）:
  Week 1–2: セキュリティ審査票提出・受領 + DPA締結
  Week 3:   SSO設定（IdP連携）+ IP制限設定 + SIEM設定
  Week 4:   専有インスタンス構築（Option Bの場合）
  Week 5–6: パイロット部署での検証（PoC）
  Week 7:   本番環境へのデプロイ + CAB承認
  Week 8:   全社展開 + 管理者トレーニング

  ※ BYOK利用の場合: Week 2–3に鍵設定を追加
  ※ 顧客GCPデプロイの場合: +4週間
```

4.8 ブラウザ拡張 技術アーキテクチャ詳細

> §4.5で概要を述べたブラウザ拡張の、Phase A/B/C それぞれの技術仕様。

Phase A: Observation Mode（既存SaaSと並行動作）

```
■ 技術スタック
  ・Chrome Extension Manifest V3
  ・Content Script: DOM観察 + 操作ログ収集
  ・Background Service Worker: ログバッチ送信 + トークン管理
  ・Storage API: ローカルキャッシュ（オフライン対応）

■ 権限（permissions）
  ・activeTab: 現在のタブのDOM読み取り
  ・storage: ローカルデータ保存
  ・identity: OAuth認証（SaaS接続）
  ・alarms: 定期バッチ送信

■ データフロー
  [SaaS画面] → Content Script（MutationObserver + click/input listener）
    → Background Service Worker（バッチキュー: 30秒ごと or 100イベント）
    → POST /api/v1/ingestion/browser-events
    → Supabase（browser_events テーブル）

■ 収集データ
  ・ページURL（SaaS特定）
  ・クリック座標 + 対象要素のセレクタ
  ・入力フィールドの変更（値はハッシュ化、PII検出時はマスク）
  ・ページ遷移パターン
  ・操作タイムスタンプ

■ セキュリティ
  ・パスワードフィールドは一切収集しない（type="password" 除外）
  ・PII自動検出（regex + LLM判定）→ マスク処理
  ・収集データはTLS 1.3で暗号化送信
  ・ユーザーが収集を一時停止/再開できるトグル
```

Phase B: UX再構築モード（業務フロー最適化 + SaaS画面統合）

Phase Bの本質は「UIの見た目を変える」ことではなく、**業務フロー自体を再設計**すること。
BPOの操作ログとナレッジから非効率パターンを検出し、クライアント固有の最適UXを構築する。

■ UX再構築の6つの柱

1. 画面統合（Multi-Screen → Single-Screen）
   ・BPO操作ログから「画面Aで入力→画面Bで確認→画面Cで承認」パターンを検出
   ・3画面遷移を1画面に統合したカスタムUIを自動生成
   ・例: kintoneの日報入力（3画面遷移）→ 統合フォーム（1画面、30秒短縮）

2. 承認フロー自動化
   ・社長の判断基準（decision_rules）をリアルタイム適用
   ・Confidence × Impact マトリクスで自動承認/要確認/要エスカレーションを判定
   ・例: 5万円以下の経費精算 → 社長ルールに合致すれば自動承認

3. 入力順序最適化
   ・業務フローの自然な順序に並び替え（SaaSのDB都合の順序ではなく）
   ・頻度の高いフィールドを上部に配置
   ・条件分岐: 前の入力値に応じて次のフィールドを動的表示

4. フィールド最適化
   ・不要フィールドの非表示（使用率5%以下を自動検出）
   ・必須フィールドの追加（社内ルールで必要だがSaaSに存在しないもの）
   ・デフォルト値の自動設定（過去パターンから推定）

5. コンテキスト連動（Auto-Prefill）
   ・前工程のデータを自動プリフィル
   ・例: 見積書作成時に顧客情報・過去単価・直近の商談メモを自動挿入
   ・関連ナレッジの自動サジェスト（入力中にリアルタイム）

6. リアルタイムバリデーション
   ・社長ルール（decision_rules）ベースの即時チェック
   ・例: 値引率15%超 → 警告「社長承認が必要です（ルールR-042）」
   ・業界規制チェック（例: 建設業の法定福利費率）

■ UXパターン検出エンジン

入力: BPO操作ログ（Phase A で蓄積）
処理:
  1. 操作シーケンス分析（画面遷移パターンの頻度集計）
  2. 非効率パターン検出:
     - 同一データの重複入力（duplicate_input_rate > 10%）
     - 3画面以上の遷移を要する操作（multi_screen_ops）
     - 入力→取消→再入力の繰り返し（correction_rate > 15%）
     - 未使用フィールド（field_usage_rate < 5%）
  3. UX改善提案の自動生成（LLM + テンプレート）
  4. 優先度スコア: time_saved × frequency × error_reduction

出力: UXRedesignProposal（統合先画面・フィールド配置・フロー定義）

■ UXテンプレート（業種別最適フロー定義）

共通テンプレート:
- 経費精算フロー（入力→自動承認判定→完了）
- 日報・週報入力（テンプレ選択→差分入力→提出）
- 顧客対応記録（顧客自動検出→対応内容→ネクストアクション）

業種別テンプレート:
- 建設業: 工事日報（天候→作業内容→安全確認→写真→原価）
- 製造業: 生産実績入力（ロット→数量→不良→設備状態）
- 歯科: 診療記録（患者→処置→材料→次回予約）

テンプレートカスタマイズ:
- 企業固有ルールの上書き（decision_rules連動）
- フィールド追加/削除/並び替え
- 条件分岐ロジックの追加

■ A/Bテスト機構（元UX vs 再構築UX）

設計:
・対象社員をランダムに2群に分割（A群: 元SaaS UX / B群: 再構築UX）
・最小2週間、各群最低5名で統計的有意性を確保

計測指標:
・task_completion_time: タスク完了時間（主指標）
・error_rate: 入力ミス率
・correction_count: 修正回数
・user_satisfaction: 5段階アンケート（週次）
・support_request_count: 問い合わせ数

判定基準:
・completion_time 10%以上改善 かつ error_rate 悪化なし → 再構築UX採用
・completion_time 改善なし or error_rate 5%以上悪化 → ロールバック
・中間判定: 1週間時点で error_rate 10%以上悪化 → 即時ロールバック

段階展開:
・Phase B-1: パワーユーザー3名で試験（1週間）
・Phase B-2: 部署単位で展開（2週間）
・Phase B-3: 全社展開（判定基準クリア後）

■ 技術的実装（UX再構築の手段）

Shadow DOM によるコンポーネント注入:
・既存SaaS画面に Shadow DOM を追加
・Shadow DOM内にシャチョツーの再構築UIを描画
・CSS Isolation: Shadow DOM境界でスタイル競合を完全防止

注入パターン（UX再構築の粒度に応じて選択）:
Pattern 1: フルページ再構築
→ SaaS画面全体を再構築UIで置換（画面統合時）
→ 元SaaSのAPIは裏で引き続き呼び出し（データ整合性維持）

Pattern 2: パーシャル再構築
→ フォーム部分のみ再構築（入力順序最適化・フィールド最適化時）
→ SaaS画面のナビゲーション等はそのまま

Pattern 3: オーバーレイ拡張
→ 承認フロー自動化・バリデーション・サジェスト等の機能追加
→ SaaS画面の上にレイヤーとして追加

状態同期:
・MutationObserver でSaaS画面のDOM変更を監視
・変更検知 → シャチョツーDBの対応レコードを更新
・双方向: シャチョツーUIからの操作 → SaaS画面のDOM操作（API経由を優先）

セキュリティ:
・CSP（Content Security Policy）対応: nonce-based script injection
・XSS防止: Shadow DOM内のスクリプトは信頼済みソースのみ
・iframe sandboxing: 外部コンテンツはsandboxed iframeで表示

Phase C: Replacement Mode（SaaS完全置換）

```
■ URL インターセプト
  ・declarativeNetRequest API でSaaS URLをインターセプト
  ・例: app.kintone.com/* → shachotwo.app/replace/kintone/*
  ・段階的カットオーバー（機能単位で切り替え）
    C-1: 閲覧系のみ置換（リスト表示、詳細表示）
    C-2: 作成・更新も置換
    C-3: 全機能置換 + データ移行完了確認
    C-4: SaaS解約

■ オフライン対応
  ・Service Worker でキャッシュ戦略を実装
  ・Cache-first: 静的アセット（CSS/JS/画像）
  ・Network-first: データAPI（オフライン時はキャッシュフォールバック）
  ・Background Sync: オフライン中の操作を同期キューに保存

■ データ移行
  ・SaaS API経由で全データを取得 → シャチョツーDBにインポート
  ・差分同期: 移行期間中はSaaS側の変更を定期同期（15分間隔）
  ・整合性チェック: レコード数・チェックサム比較
  ・ロールバック保証: 移行後30日間はSaaSデータを保持
```

4.9 SaaS Dependency Graph 詳細スキーマ

> SaaS間のデータフローを有向グラフで表現し、
> 解約順序の最適化・循環依存の検出を行う。

データモデル

```
from enum import Enum
from pydantic import BaseModel
from typing import Optional
from datetime import date

class SaaSCategory(str, Enum):
    CRM = "crm"                     # 顧客管理
    ACCOUNTING = "accounting"        # 会計
    HR = "hr"                        # 人事・労務
    COMMUNICATION = "communication" # コミュニケーション
    PROJECT = "project_mgmt"         # プロジェクト管理
    STORAGE = "storage"              # ファイル管理
    MARKETING = "marketing"          # マーケティング
    CUSTOM = "custom"                # その他

class Criticality(str, Enum):
    CRITICAL = "critical"       # 停止不可（会計、基幹）
    IMPORTANT = "important"     # 代替手段あり
    NICE_TO_HAVE = "nice"       # なくても困らない

class DataFlowType(str, Enum):
    MASTER = "master"           # マスタデータ連携
    TRANSACTION = "transaction" # トランザクション連携
    NOTIFICATION = "notification"  # 通知連携
    FILE = "file"               # ファイル連携

class SaaSNode(BaseModel):
    name: str                               # SaaS名
    category: SaaSCategory
    monthly_cost: int                       # 月額コスト（円）
    user_count: int                         # 利用ユーザー数
    criticality: Criticality
    replacement_difficulty: float           # 0.0〜1.0（§4.7の算出式）
    contract_end: Optional[date]            # 契約満了日
    notice_required_days: int = 30          # 解約予告日数
    data_volume_gb: Optional[float]         # 保存データ量

class DataFlowEdge(BaseModel):
    from_saas: str                          # データ送信元
    to_saas: str                            # データ受信先
    data_type: DataFlowType
    frequency: str                          # realtime/daily/weekly/monthly
    volume_monthly: int                     # 月次データ件数
    is_bidirectional: bool = False          # 双方向か
    can_be_broken: bool = True              # 切断可能か（False=解約順序の制約）
    migration_effort_hours: Optional[float]  # 移行工数（時間）

class SaaSDependencyGraph(BaseModel):
    company_id: str
```

```
    nodes: dict[str, SaaSNode]              # SaaS名 → ノード
    edges: list[DataFlowEdge]               # データフロー一覧
```

循環依存の検出・解決

```
def detect_cycles(graph: SaaSDependencyGraph) -> list[list[str]]:
    """
    有向グラフの循環を検出（Tarjanの強連結成分アルゴリズム）。
    循環がある場合、解約順序のトポロジカルソートが不可能。
    """
    # Tarjan's SCC algorithm
    # 循環を含むSCCを返す
    ...

def resolve_cycle(cycle: list[str], edges: list[DataFlowEdge]) -> str:
    """
    循環内で最もコストの低いエッジを切断して循環を解消。
    切断コスト = migration_effort_hours × (1 if can_be_broken else 10)
    """
    cycle_edges = [e for e in edges
                   if e.from_saas in cycle and e.to_saas in cycle]
    min_cost_edge = min(cycle_edges,
                        key=lambda e: (e.migration_effort_hours or 8)
                        * (1 if e.can_be_broken else 10))
    return f"Cut: {min_cost_edge.from_saas} → {min_cost_edge.to_saas}"
```

解約順序の最適化

```
def optimal_cancellation_order(graph: SaaSDependencyGraph) -> list[str]:
    """
    トポロジカルソート + コスト最適化で解約順序を算出。
    原則: 依存されていないSaaSから先に解約。
    同順位の場合: 月額コストが低い方を先に（リスク最小化）。
    """
    # 1. 循環を検出・解消
    # 2. トポロジカルソート
    # 3. 同一レベル内でcost昇順ソート
    # 4. 各SaaSの解約可能月を法的チェック（notice_required_days）と突合
    ...
```

4.10 Data Portability / Export API設計

> 顧客がいつでもデータを持ち出せることを保証する。
> ロックイン防止 = 信頼 = 長期契約。

エクスポート対象

データ種別	含まれる内容	形式
ナレッジ	全ナレッジアイテム + メタデータ + 関係グラフ	JSON / CSV
業務フロー	フロー定義 + ステップ + ボトルネック	JSON / Mermaid
意思決定ルール	ルール定義 + 条件 + 例外	JSON / CSV
ゲノム	9次元モデル全データ	JSON
BPOワークフロー	ワークフロー定義 + 実行履歴サマリー	JSON
監査ログ	直近1年分の全操作ログ	JSON / CSV

Export API

```
■ エクスポートジョブ作成
  POST /api/v1/export/request
  Body: {
    "format": "json" | "csv" | "sql_dump",
    "scope": ["knowledge", "flows", "rules", "genome", "bpo", "audit"],
    "date_range": {"from": "2025-01-01", "to": "2026-03-12"},  // optional
    "encryption_password": "..."  // optional, 指定時AES-256-GCMで暗号化
  }
  Response: { "job_id": "xxx", "estimated_minutes": 5 }

■ ジョブ状態確認
  GET /api/v1/export/{job_id}/status
  Response: {
    "status": "pending" | "processing" | "completed" | "failed",
    "progress_percent": 75,
    "file_size_mb": 120
  }

■ ダウンロード
  GET /api/v1/export/{job_id}/download
  Response: Binary (ZIP, AES-256-GCM暗号化済み)
  ※ ダウンロードリンクの有効期限: 24時間
  ※ 全エクスポート操作は audit_logs に記録
```

セキュリティ

```
・エクスポート実行には CEO or Manager ロールが必要
・エクスポートファイルは AES-256-GCM で暗号化
・パスワードは別チャネル（LINE）で通知
・不正検出: 24時間以内に3回以上のエクスポート → アラート
・監査: 全エクスポート操作をログ記録（誰が・いつ・何を・どのくらい）
```

---

7. 3編の全体フロー（まとめ）

```
001_ブレイン編（コンサルを潰す）
  ↓ ナレッジが溜まる
  ↓ 業務フロー・判断基準・暗黙知が構造化される
  ↓
002_BPO編（業務代行・人材紹介を潰す）
  ↓ ナレッジが「実行」に変換される
  ↓ 実行データ（成功/失敗/ボトルネック）が溜まる
  ↓
003_自社システム編（SaaSを潰す）
  ↓ 実行データが「システム要件」に変換される
  ↓ 個社専用システムが自動構築される
  ↓
  ↓ ── そしてデータネットワーク効果 ──
  ↓
  ↓ 全社のナレッジ × 実行データ × システムパターン
  ↓ → 匿名化 → 業界標準化 → 新規企業に還元
  ↓ → 使えば使うほど、全員が賢くなる
  ↓ → これが競合が追いつけない最大の堀（Moat）

最終形: シャチョツー = 中小企業の「IT部門 + コンサル + BPO」のオールインワン
        月3-8万で、年間1,000万以上のコスト削減を実現
```